

《算法设计技巧与分析》

第1章 算法分析基本概念

朱友文

zhuyw@nuaa.edu.cn

课程QQ群



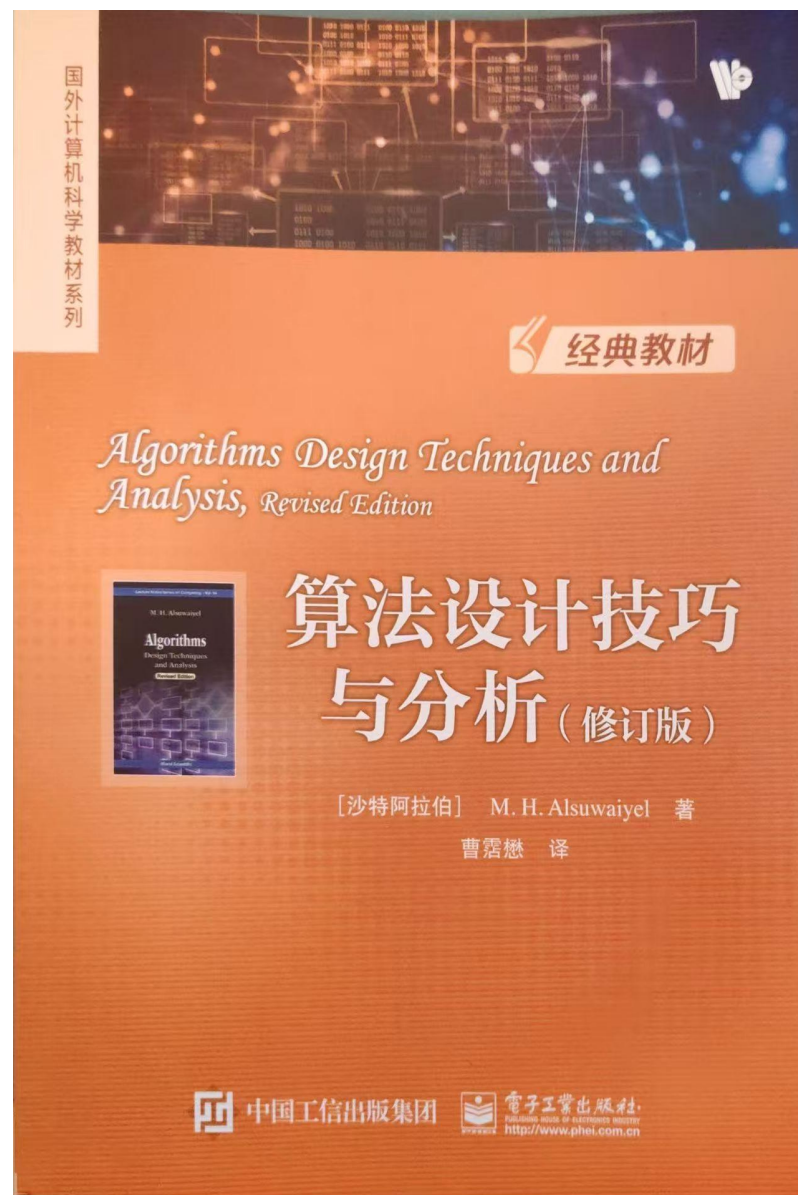
- 1-3周，每周三上午10:30-12:15，牧星楼T2301
- 4-10周，每周四下午14:00-15:45，牧星楼T2401
- 1-10周，每周五上午10:30-12:15，，牧星楼T2401
- 考核方式：闭卷笔试
- 最终成绩：70%期末考试+30%平时成绩
- 平时成绩：平时作业+随堂测试

教材

《算法设计技巧与分析》

M. H. Alsuwaiyel著,

电子工业出版社



什么是算法？

- 完成某一任务的方法
- 算法的特征：
 - 有输入、有输出
 - 确切性
 - 可行性
 - 有穷性
- 数据结构与算法

引言：衡量算法性能的标准

- 衡量算法性能一般有以下几个标准
 - 正确性
 - 易读性
 - 健壮性
 - 算法的时间和空间性能：高效率和低存储空间

我们这里主要讨论算法的**时间**和**空间**性能，并以此作为衡量算法性能的重要标准。而且我们主要侧重于**时间**方面。

1.3 二分搜索

- 假定元素满足：线序集合
- $A[1...n]$ 中有 x 吗？从头到尾的扫描，依次比较：顺序搜索（适合无序的集合）

算法 1.1 LINEARSEARCH

输入： n 个元素的数组 $A[1...n]$ 和元素 x 。

输出：如果 $x = A[j]$, $1 \leq j \leq n$, 则输出 j , 否则输出 0。

1. $j \leftarrow 1$
2. **while** $(j < n)$ **and** $(x \neq A[j])$
3. $j \leftarrow j + 1$
4. **end while**
5. **if** $x = A[j]$ **then return** j **else return** 0

1.3- (例) 线性查找的时间评估

➤ 最小查找时间? 最好情况, $A[1]=X$

➤ 平均查找时间?

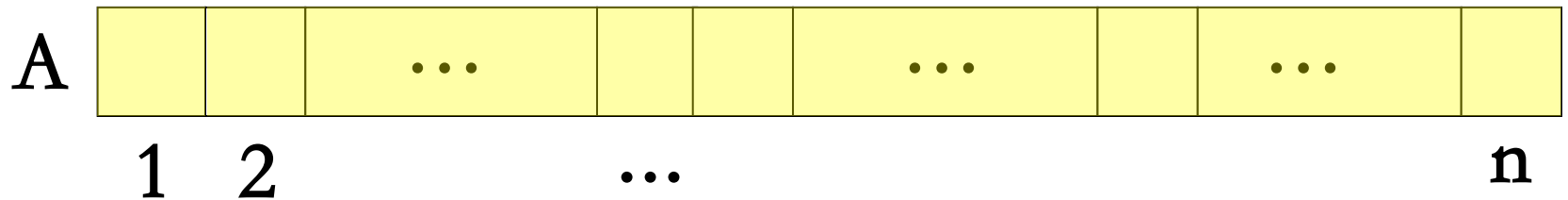
$$T_{\text{avg}}(n) = n/2$$

➤ 最大查找时间? 最坏情况.

x 不在 $A[1 \dots n]$ 或 $x = A[n]$, 复杂度为 n

1.3 二分搜索

- 有序(非降序)的集合: BinarySearch **P4**



1. 3 二分搜索及其时间复杂度

- 二分搜索

算法 1.2 BINARYSEARCH

输入： n 个元素的升序数组 $A[1 \cdots n]$ 和元素 x 。

输出：如果 $x = A[j]$, $1 \leq j \leq n$, 则输出 j , 否则输出 0。

```
1.  $low \leftarrow 1; high \leftarrow n; j \leftarrow 0$ 
2. while ( $low \leq high$ ) and ( $j = 0$ )
3.    $mid \leftarrow \lfloor (low + high) / 2 \rfloor$ 
4.   if  $x = A[mid]$  then  $j \leftarrow mid$ 
5.   else if  $x < A[mid]$  then  $high \leftarrow mid - 1$ 
6.   else  $low \leftarrow mid + 1$ 
7. end while
8. return  $j$ 
```

比较次数分析

- 最少比较一次
- 最多呢?

在while循环中, 第1次循环之后 $A[mid+1, \dots, n]$
剩余的元素个数为: $n/2$ 或者 $(n-1)/2$

也就是, $\lfloor n/2 \rfloor$

比较次数分析

接着，在第2次循环之后剩余的元素个数为：

$$\lfloor \lfloor n/2 \rfloor / 2 \rfloor$$

也就是，？？？

比较次数分析

接着，在第2次循环之后剩余的元素个数为：

$$\lfloor \lfloor n/2 \rfloor / 2 \rfloor$$

也就是， $\lfloor n/4 \rfloor = \lfloor \lfloor n/2 \rfloor / 2 \rfloor$ ？

思考：你能快速的证明上述结论吗？

比较次数分析

那么，在第j-1次循环之后剩余的元素个数为：

$$\lfloor n/2^{j-1} \rfloor$$

比较次数分析

最多比较次数时, $\lfloor n/2^{j-1} \rfloor = 1$

所以, $1 \leq n/2^{j-1} < 2$

$$2^{j-1} \leq n < 2^j$$

以2为底, $j-1 \leq \log n < j$

$$j = \lfloor \log n \rfloor + 1$$

比较次数分析

最大循环次数,
最多比较次数, $\lfloor \log n \rfloor + 1$

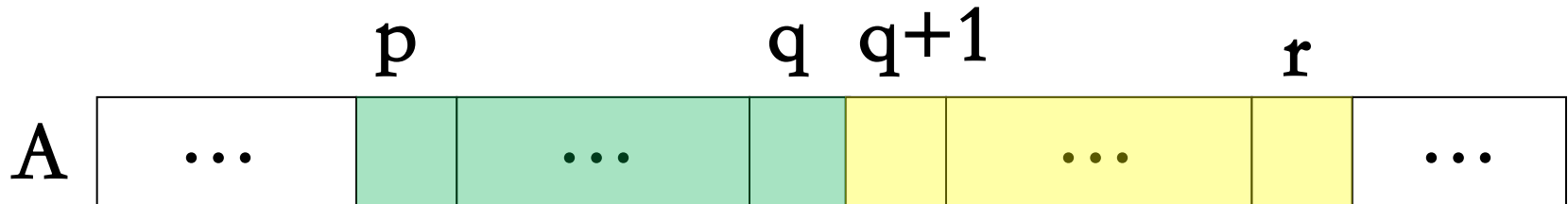
数组 $A[1\dots n]$ 中存放了 n 个升序的正整数，给定正整数 w ，设计一个 $O(n\log n)$ 的算法，判断数组 A 中是否存在两个元素的平方和等于 w （即，判断是否存在不同的 i 和 j ，能够使得 $A[i]^2 + A[j]^2 = w$ 成立）。

如果存在，返回这两个元素的下标（如果存在多组满足条件的，只需返回其中一组的下标）；如果不存在，返回“Not Exist”。



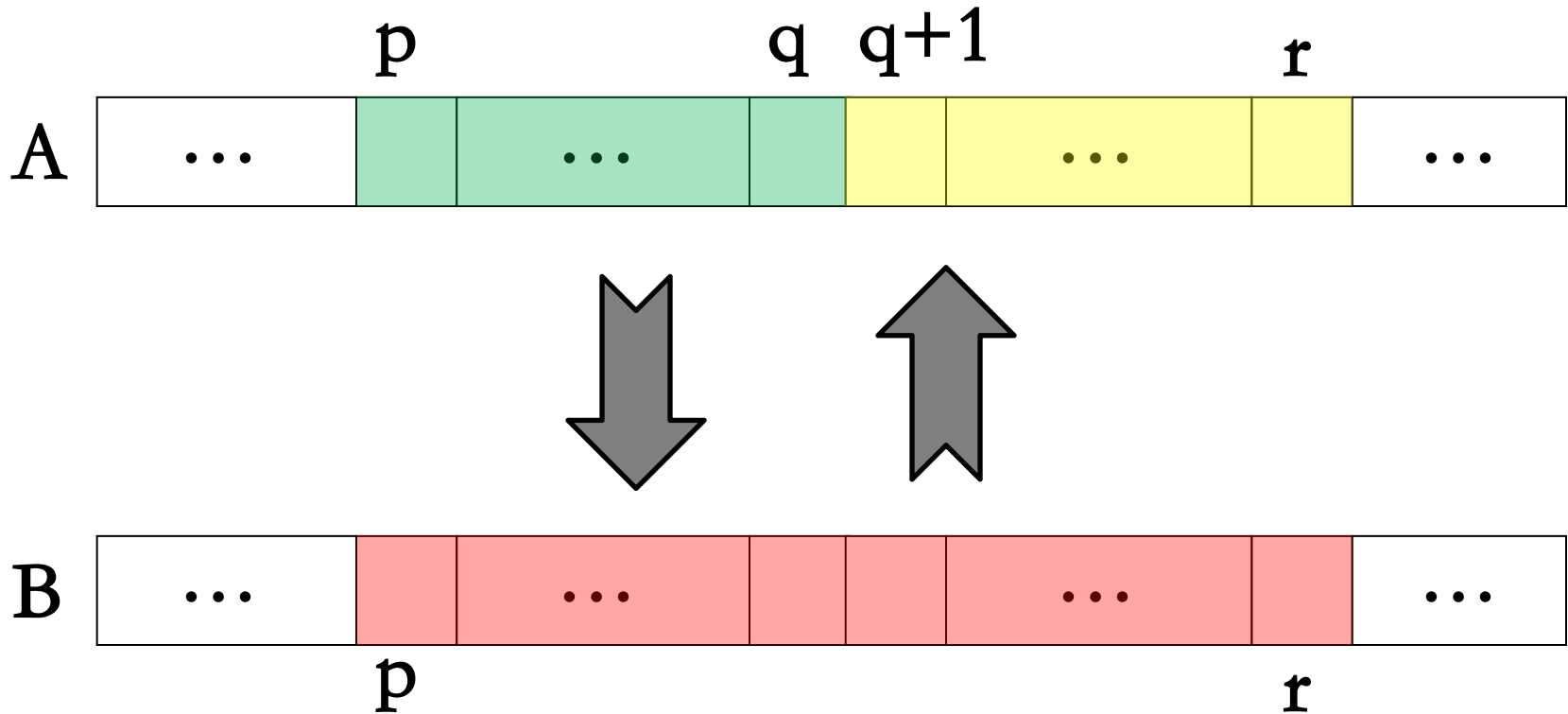
- 个人主页 zhuyw.cn
<https://faculty.nuaa.edu.cn/zhuyouwen/>
- 研究方向：低空安全、AI安全、数据安全、区块链
- 团队成员：教授1人、副教授4人
- 团队学生：博士生13人(在职的5人)，硕士20多人
- 科研项目：国家项目5项(国家重点2项)，累计科研经费1000+万元
- 论文、标准和专利：CCF A类等顶会顶刊论文50+篇

1.4 合并两个已排序的表

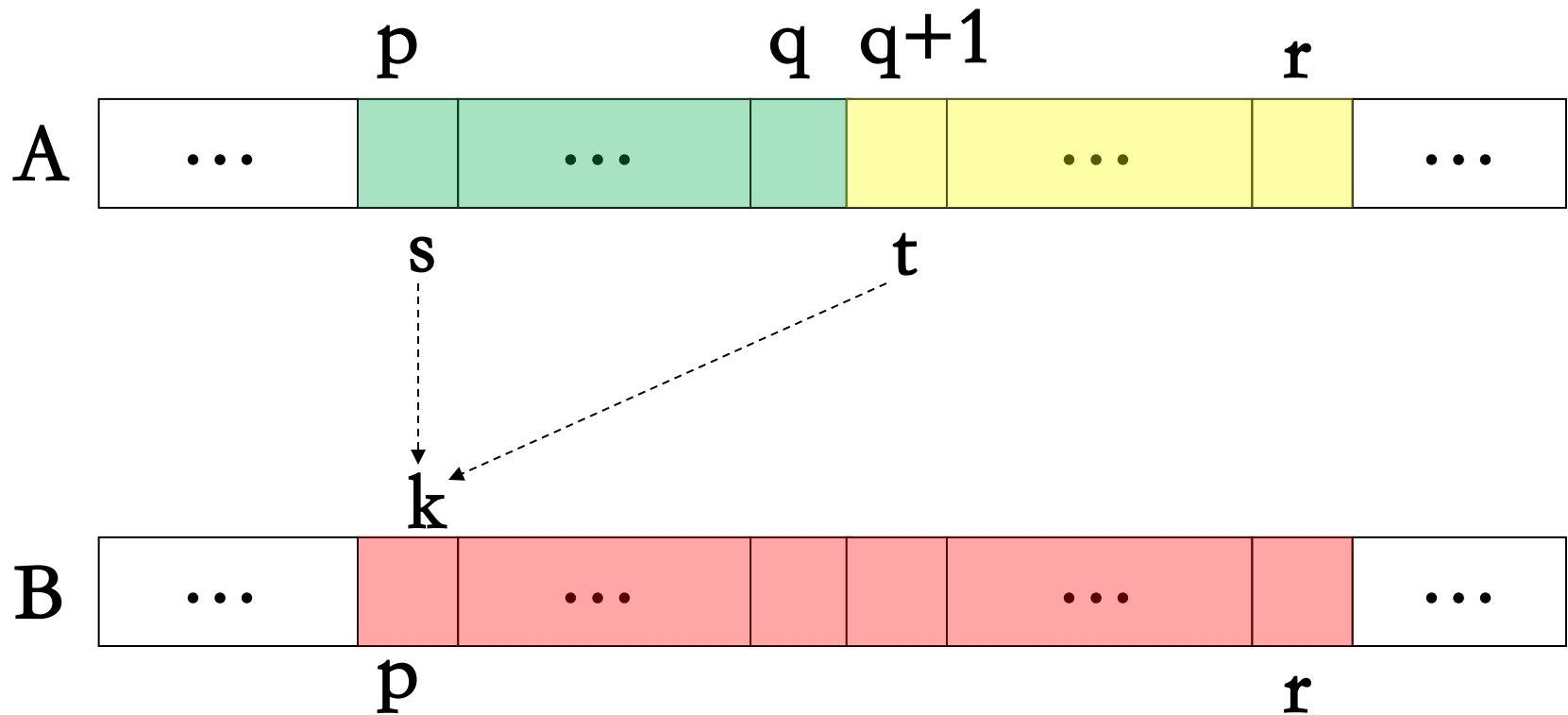


- 输入：数组 $A[p \dots q]$, $A[q+1 \dots r]$ 各自是有序的（非降序）
- 输出：使得 $A[p \dots r]$ 是有序的（非降序）

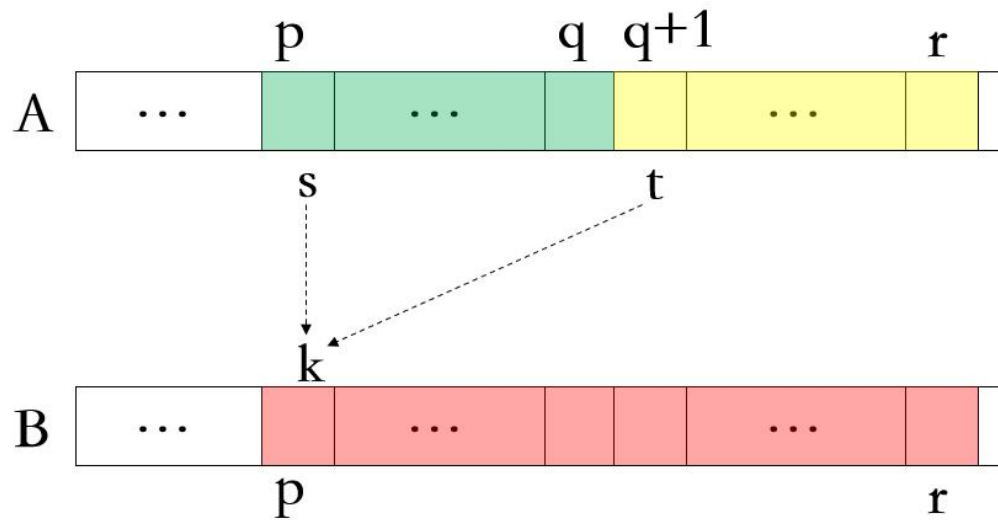
1.4 合并两个已排序的表



1.4 合并两个已排序的表



1.4 合并两个已排序的表



- 空间开销:

数组B, n

- 时间开销:

比较次数, 赋值次数

```
Merge(A[], p, q, r)
{
    s = p, t = q+1, k = p;
    while( s ≤ q 且 t ≤ r ) {
        if(A[s] > A[t]) {
            B[k] = A[t]; t++;
        } else {
            B[k] = A[s]; s++;
        }
        k++;
    }
    if(s > q) B[k...r] = A[t...r];
    else B[k...r] = A[s...q];
    A[p...r] = B[p...r];
}
```

观察结论1.1:

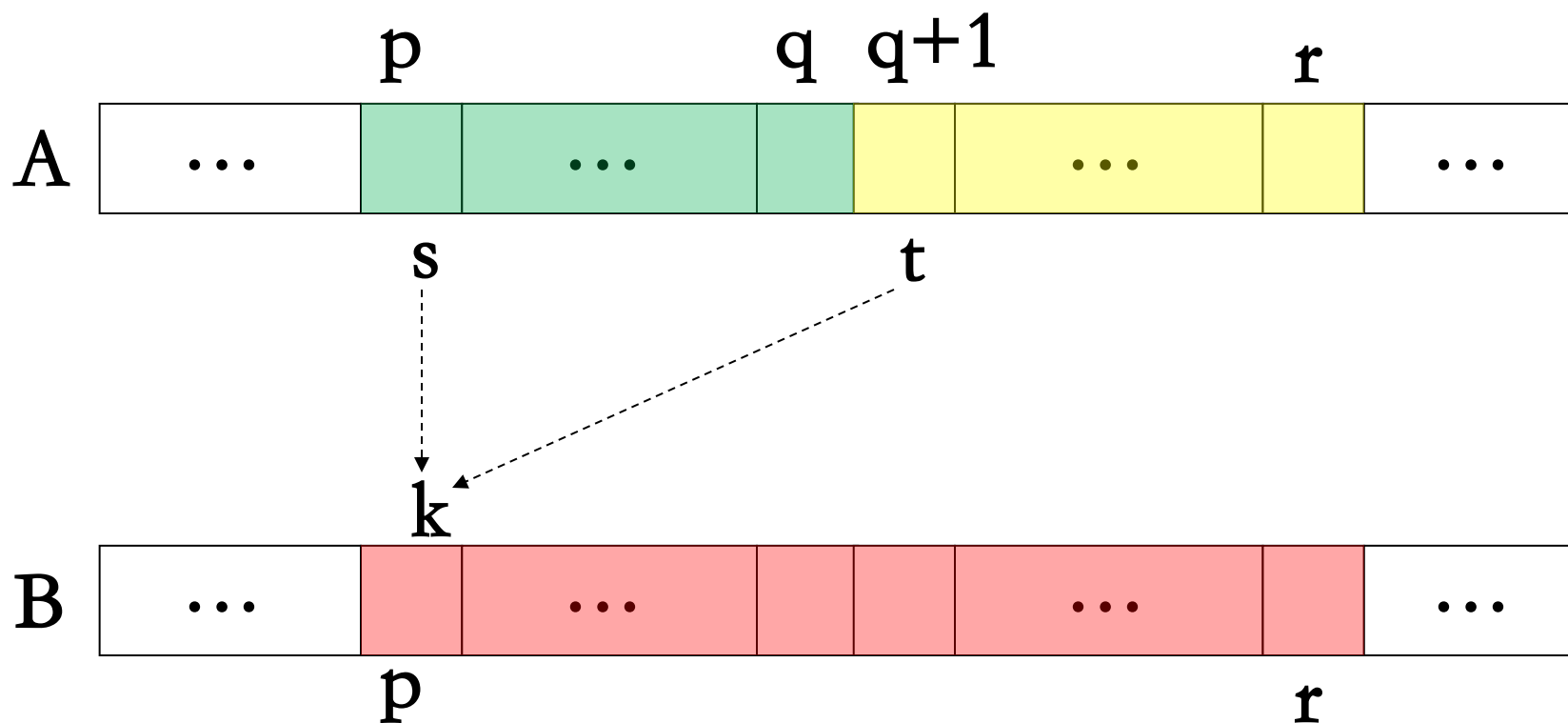
- Merge 将 $n_1=q-p+1$ 和 $n_2=r-q$ 大小的两个数组合并成一个 $n=n_1+n_2$ 的数组, ($n_1 \leq n_2$) 元素的比较次数为 n_1 到 $n-1$ 之间。
- 特例: 如果两个数组的大小为 $\lfloor n/2 \rfloor$ 和 $\lceil n/2 \rceil$, 需要比较的次数在 $\lfloor n/2 \rfloor$ 到 $n-1$ 之间。

观察结论1.2

- 使用算法Merge将两个数组合并为一个大小为 n 的有序数组，元素赋值的次数恰好是 $2n$

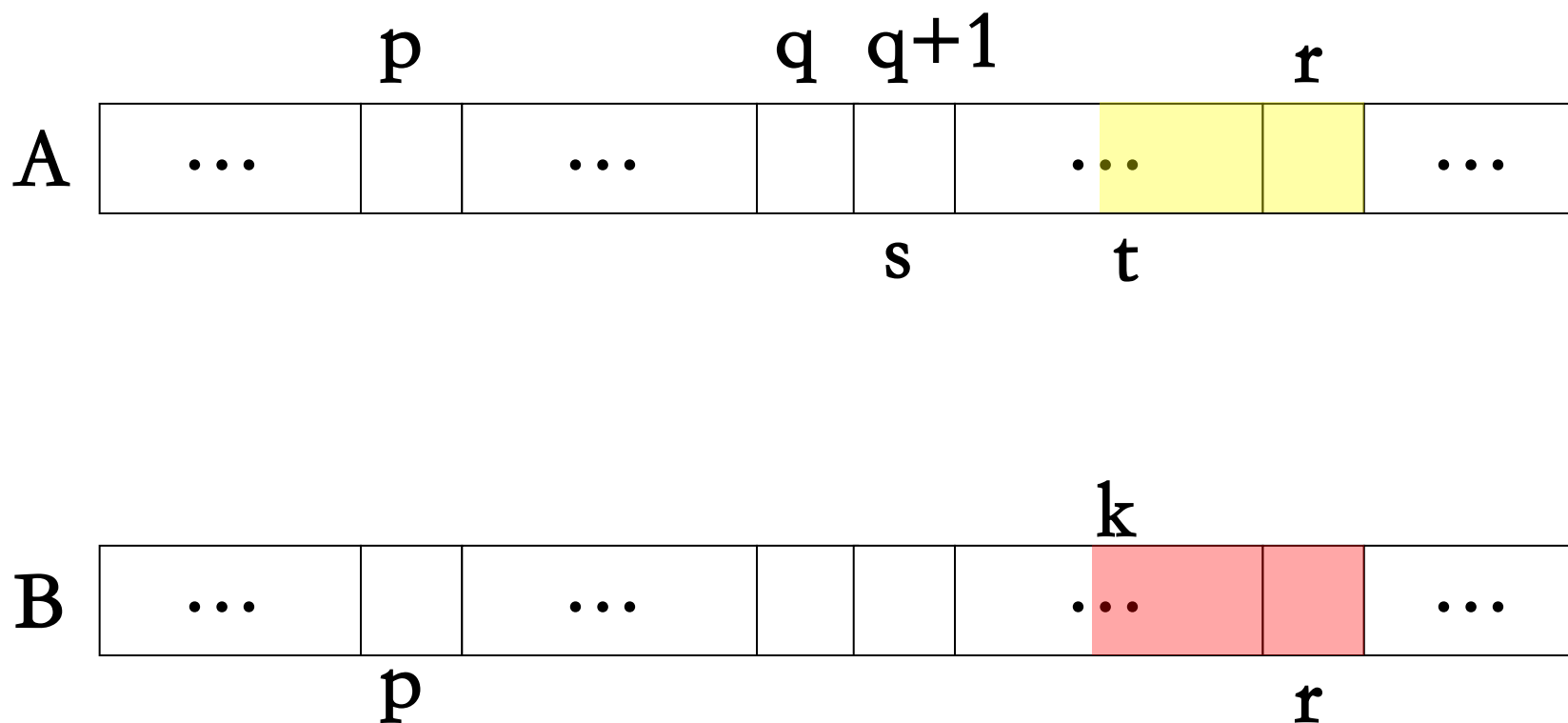
思考1

- 你能进一步改进数组合并算法 Merge 吗?



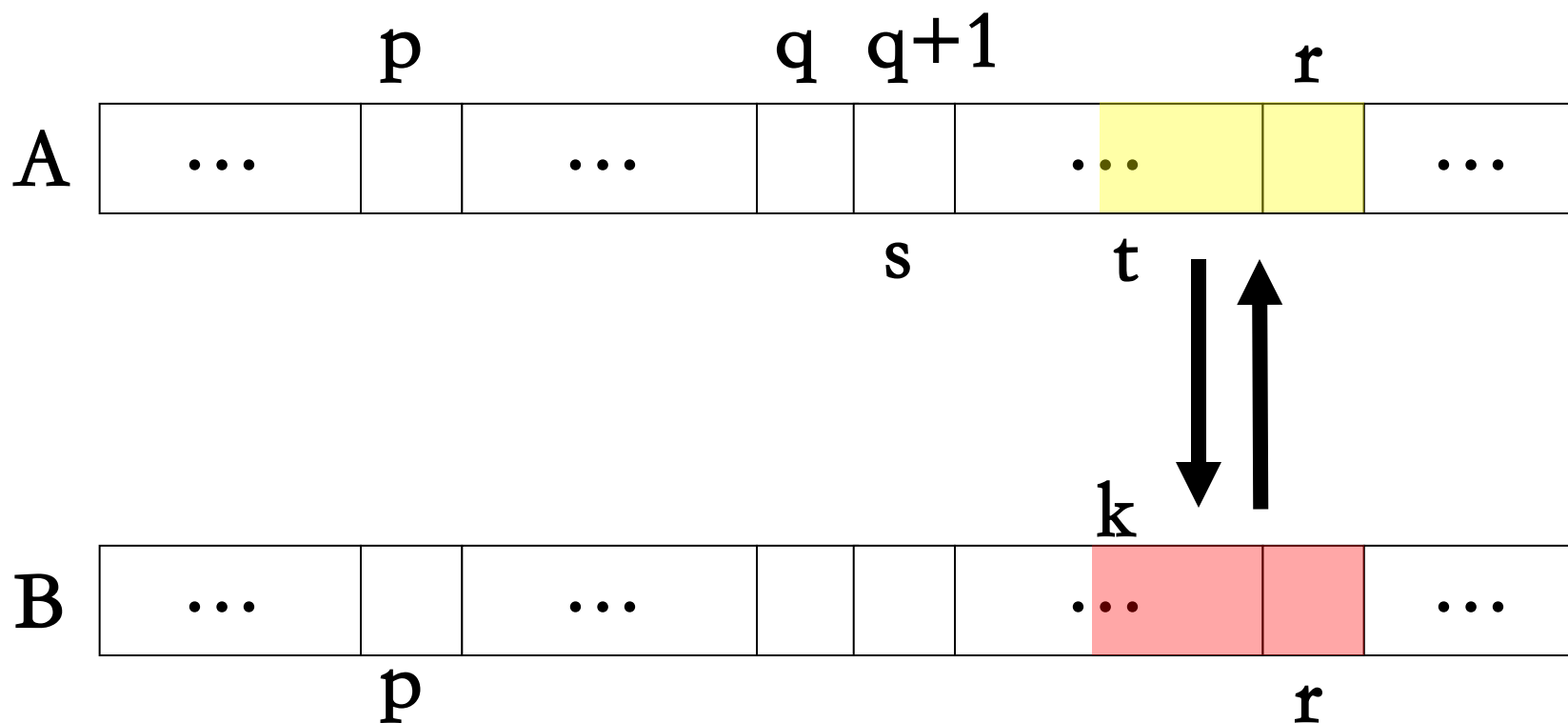
思考1

- 第1种情况



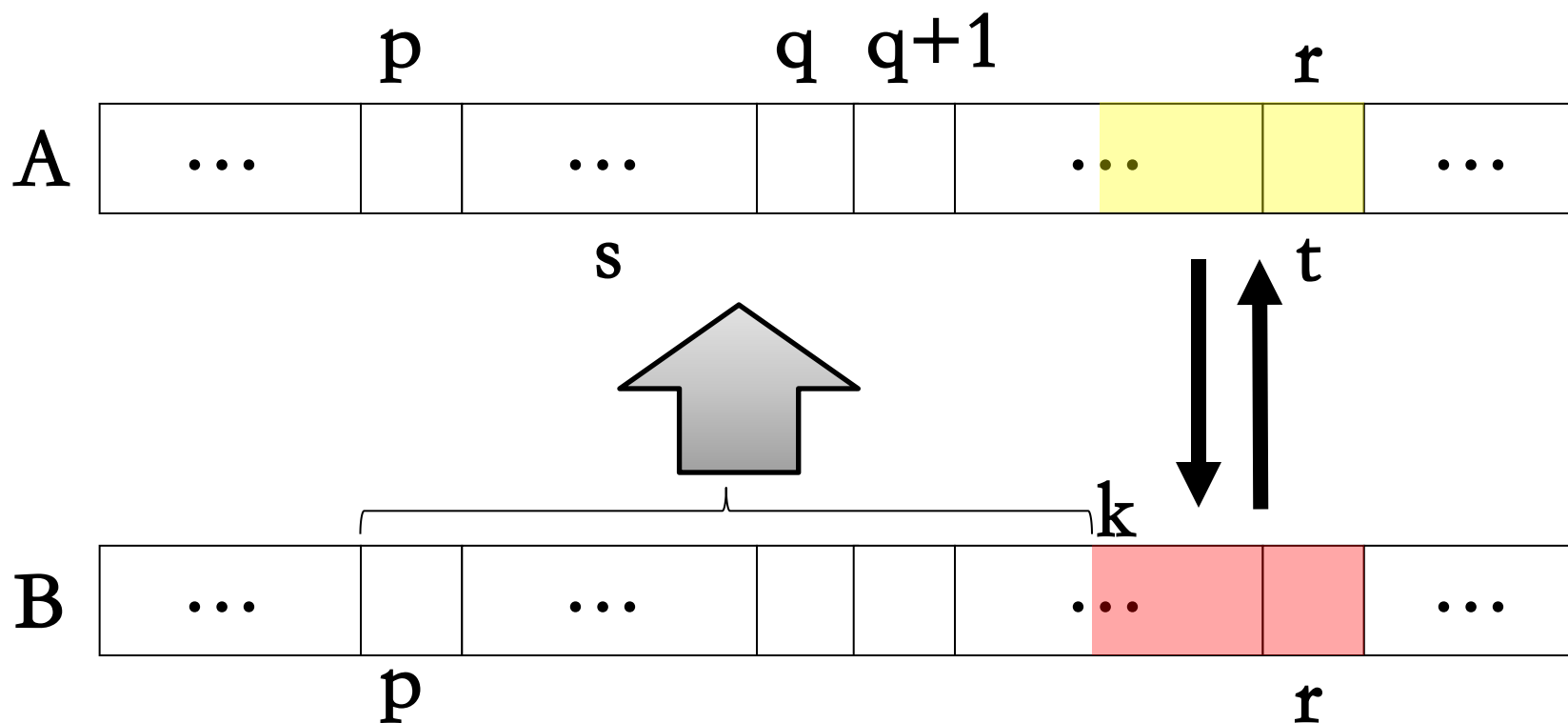
思考1

- 第1种情况



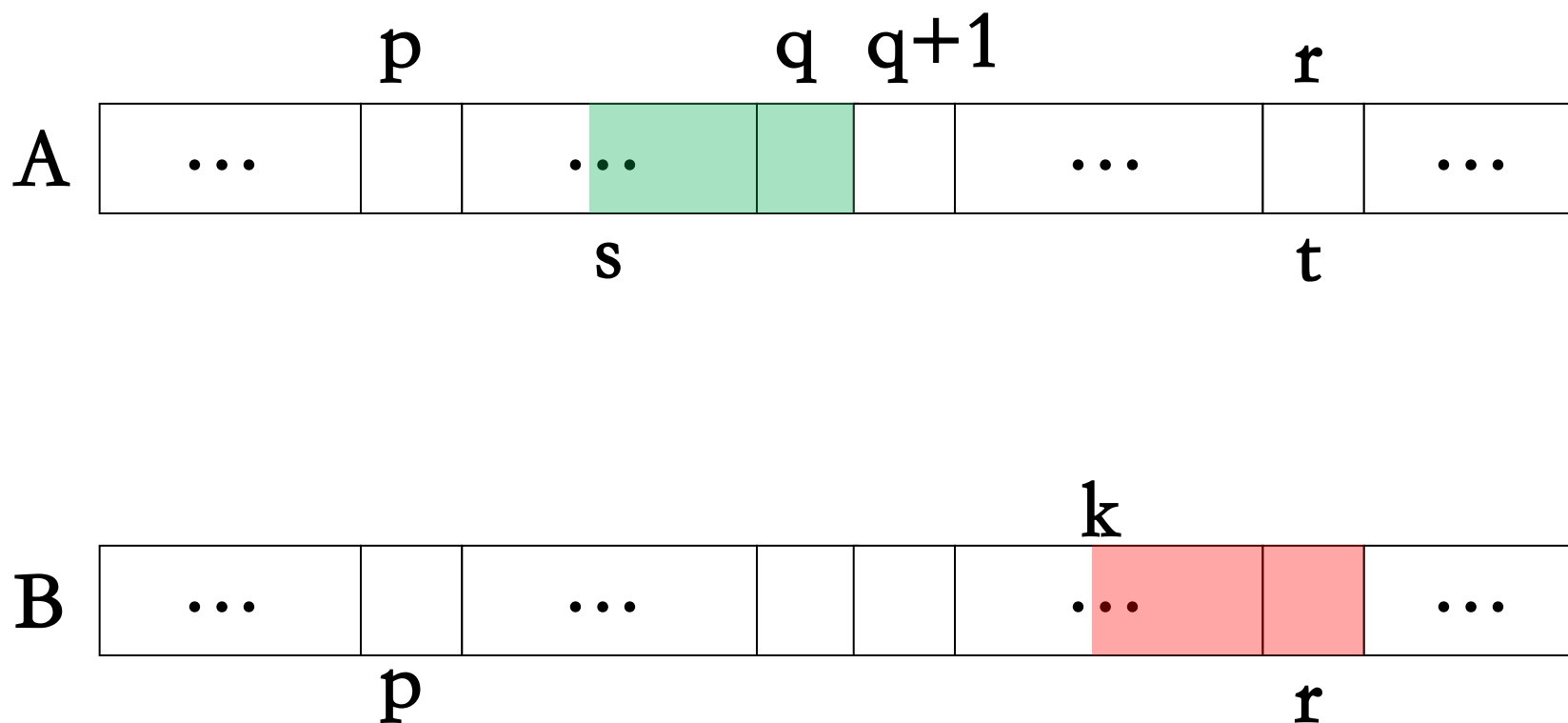
思考1

- 第1种情况



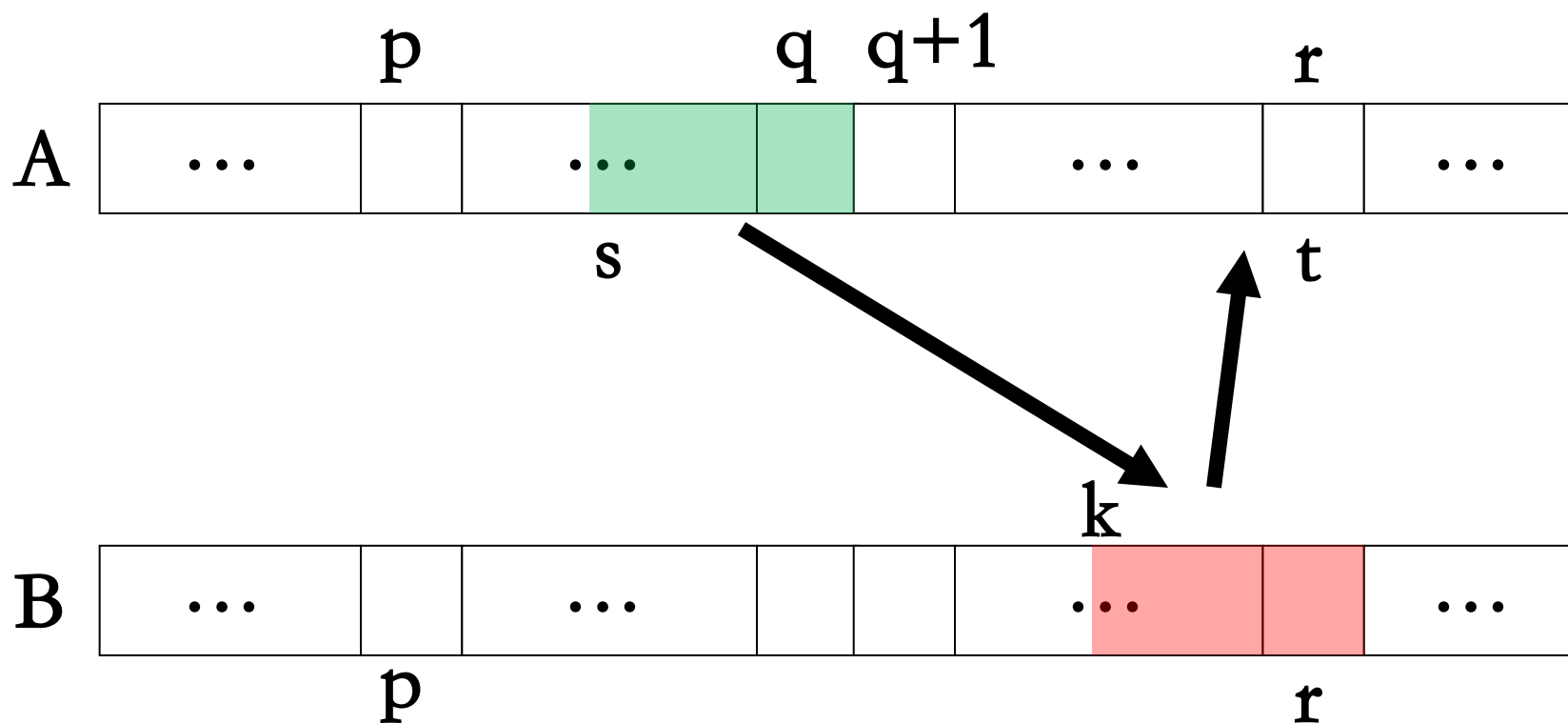
思考1

- 第2种情况



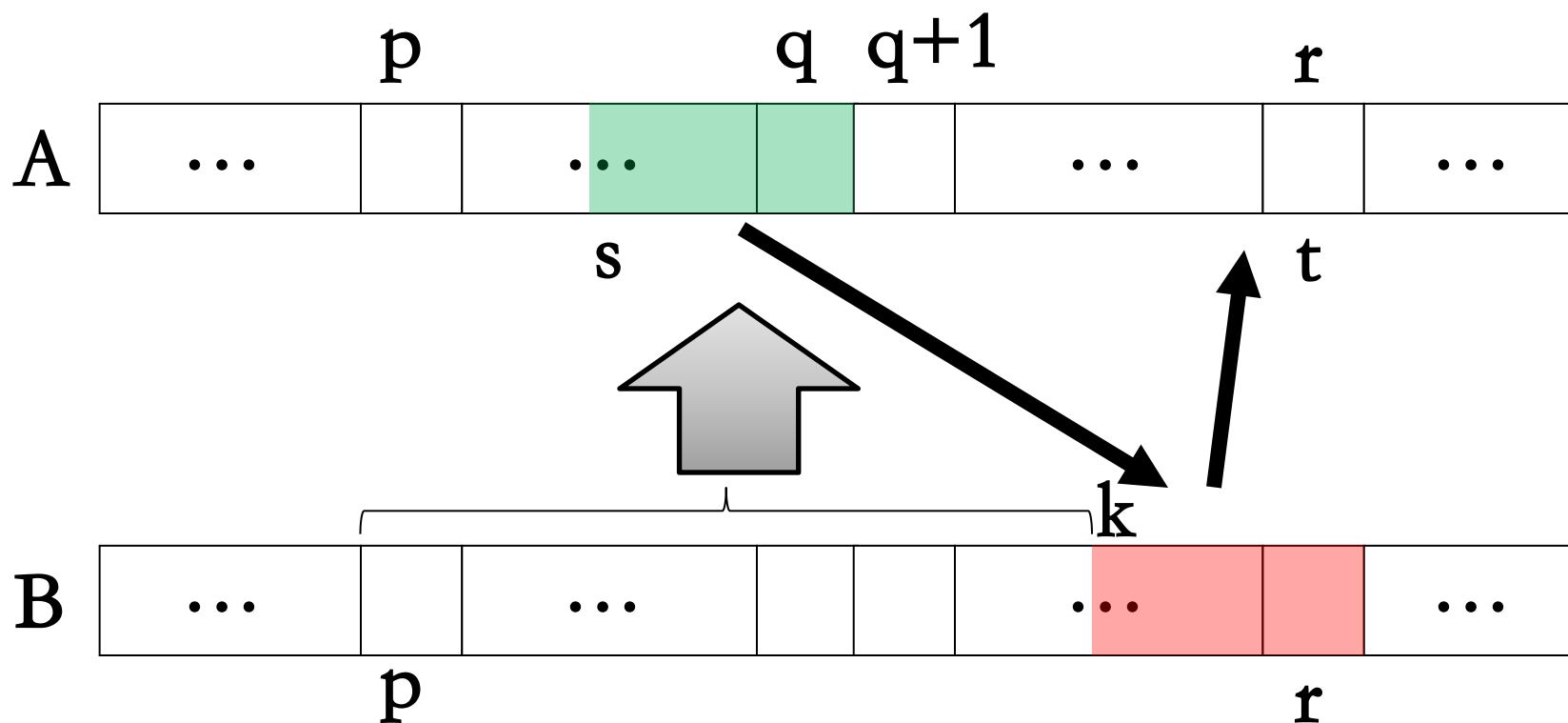
思考1

- 第2种情况



思考1

- 第2种情况

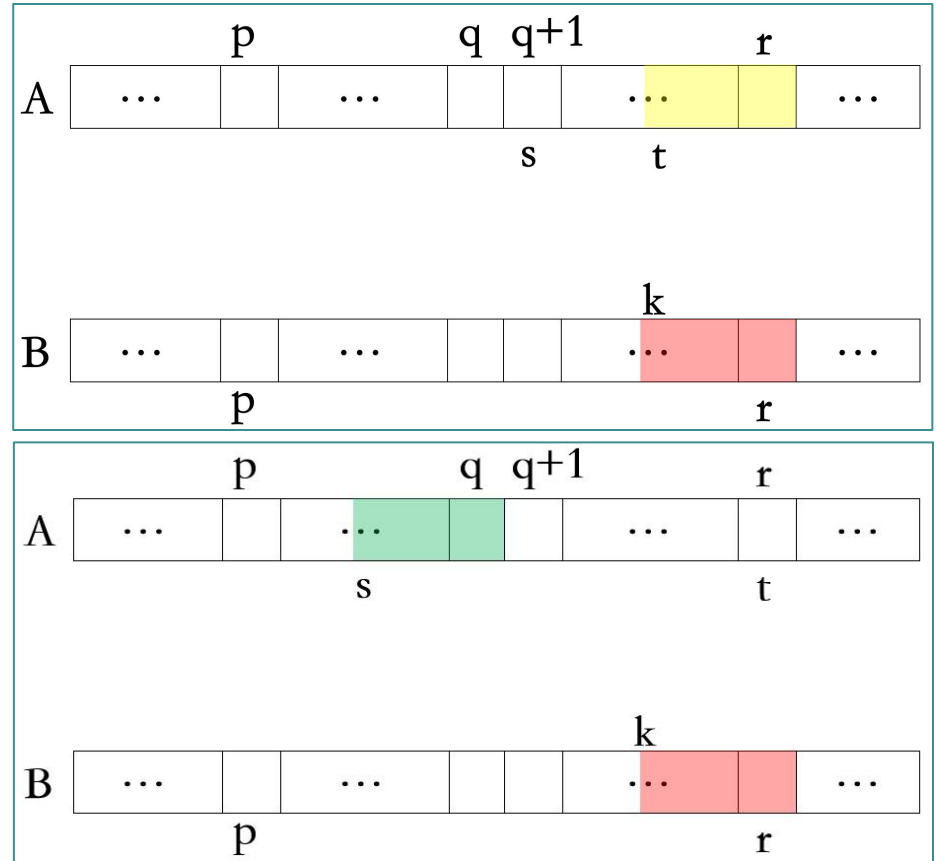


1.4 合并两个已排序的表

Merge(A[], p, q, r)

```
{  
    s = p, t = q+1, k = p;  
    while( s ≤ q 且 t ≤ r ) {  
        if(A[s] > A[t]) {  
            B[k] = A[t]; t++;  
        } else {  
            B[k] = A[s]; s++;  
        }  
        k++;  
    }
```

```
    if(s > q) B[k...r] = A[t...r];  
    else B[k...r] = A[s...q];  
    A[p...r] = B[p...r];  
}
```



```
if(s > q) A[p...k-1] = B[p...k-1];  
else { A[k...r] = A[s...q];  
       A[p...k-1] = B[p...k-1]; }
```

思考2

- 假如没有辅助数组 $B[p \cdots r]$ ，怎么办？

思考2

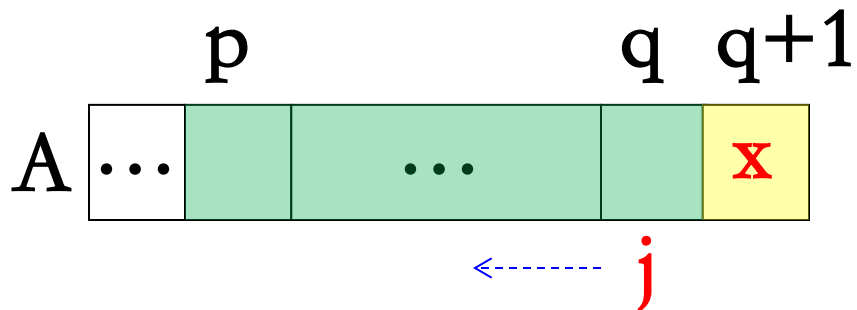
- 假如没有辅助数组 $B[p \cdots r]$ ，怎么办？

第1种情况： $r=q+1$



思考2

第1种情况: $r=q+1$



```

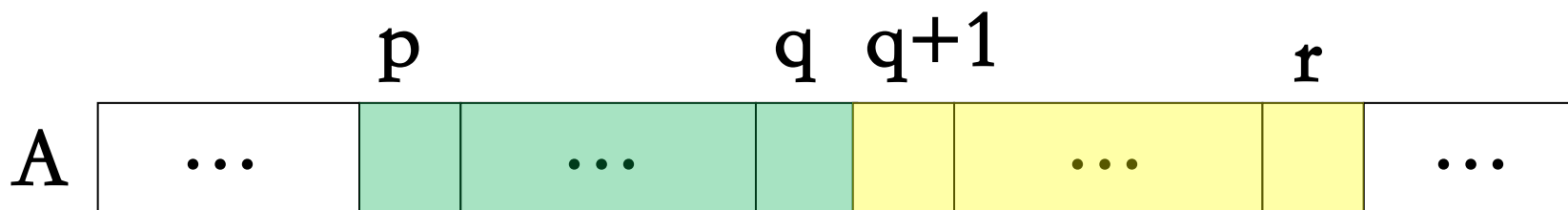
MergeY2(A[], p,q)
{
    x = A[q+1];
    j = q;
    while (p ≤ j) {
        if(A[j] > x){
            A[j+1] = A[j];
            j--; }else{break;}
    }//endwhile
    A[j+1] = x;
}

```

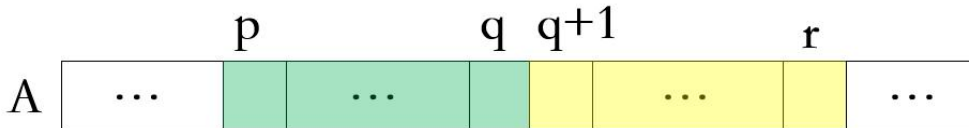
思考2

- 假如没有辅助数组 $B[p \cdots r]$ ，怎么办？

第2种情况： $r > q+1$



第2种情况: $r > q+1$



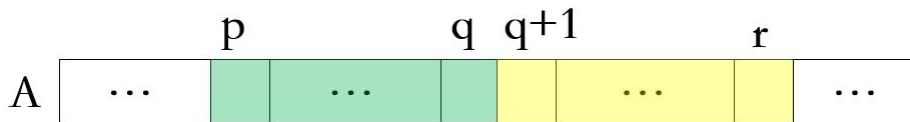
```
MergeY3(A[], p,q,r)
{
  for(i=q+1; i ≤ r; i++) {
    x = A[i];
    j = i-1;
    while (p ≤ j) {
      if(A[j] > x){
        A[ j+1] = A[j];
        j--; }else{break;}
    }//endwhile
    A[ j+1] = x;
  }//endfor
}
```

思考2

- 假如没有辅助数组 $B[p \cdots r]$, 怎么办?
- 算法开销?

• 进一步观察这个算法

第2种情况: $r > q+1$

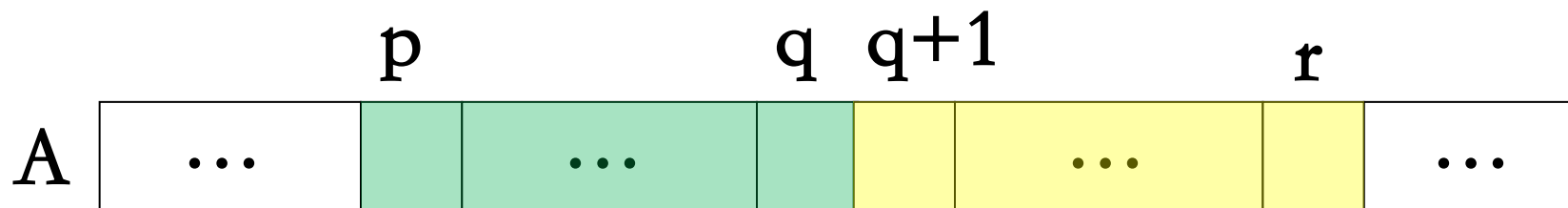


$q+1 \cdots r$ 部分乱序也可以

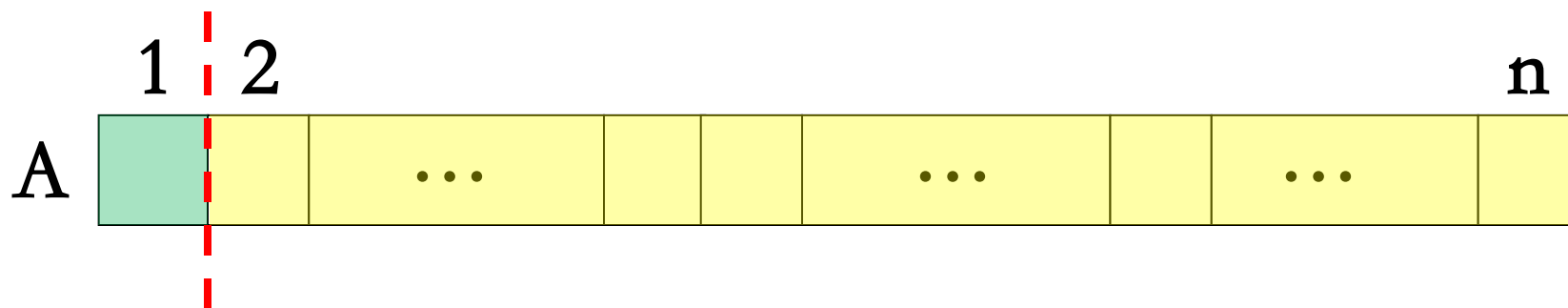
```
MergeY3(A[], p,q,r)
{
  for(i=q+1; i ≤ r; i++) {
    x = A[i];
    j = i-1;
    while (p ≤ j) {
      if(A[j] > x){
        A[ j+1] = A[j];
        j--; }else{break;}
      }//endwhile
      A[ j+1] = x;
    }//endfor
  }
```

• 进一步观察这个算法

第2种情况: $r > q+1$

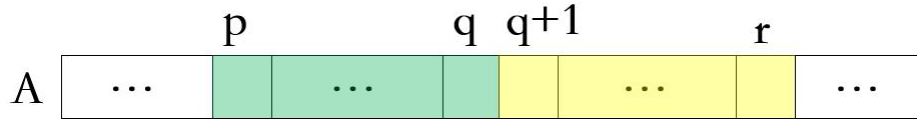


$q+1 \cdots r$ 部分乱序也可以



• 进一步观察

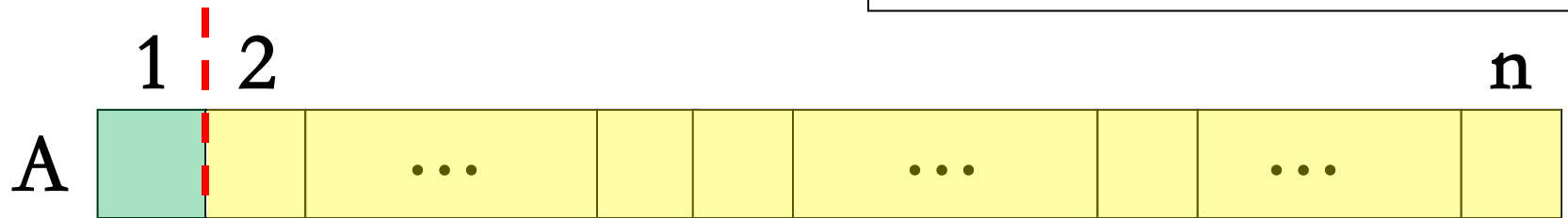
第2种情况: $r > q+1$



$q+1 \cdots r$ 部分乱序也可以

调用

MergeY3(A, 1, 1, n);



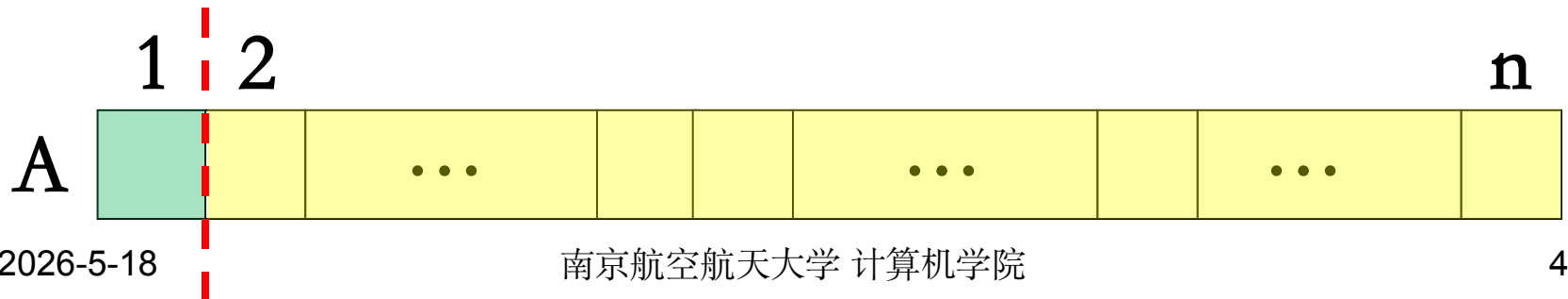
```
MergeY3(A[], p, q, r)
{
  for(i=q+1; i ≤ r; i++) {
    x = A[i];
    j = i-1;
    while (p ≤ j) {
      if(A[j] > x) {
        A[j+1] = A[j];
        j--; } else {break;}
      } //endwhile
    A[j+1] = x;
  } //endfor
}
```

```
MergeY3(A[], p,q,r)
```

```
{  
  for(i=q+1; i ≤ r; i++ ){  
    x = A[i];  
    j = i-1;  
    while (p ≤ j) {  
      if(A[j] > x){  
        A[ j+1] = A[j];  
        j--; }else{break;}  
    }//endwhile  
    A[ j+1] = x;  
  }//endfor  
}
```

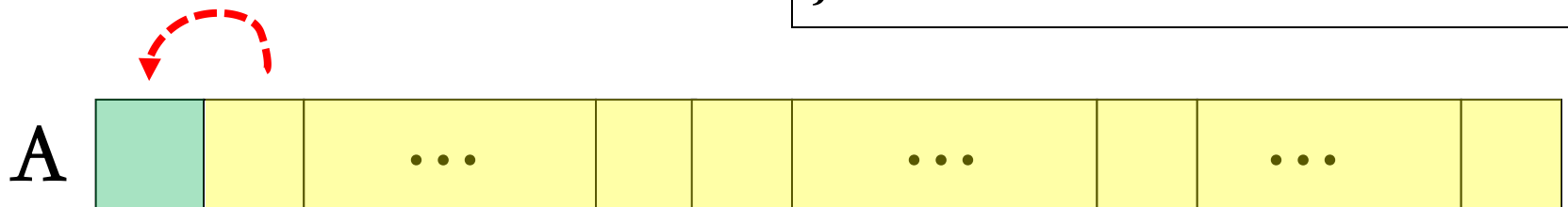
```
MergeY4(A[], n)//p=q=1,r=n
```

```
{  
  for(i=2; i ≤ n; i++ ){  
    x = A[i];  
    j = i-1;  
    while (1 ≤ j) {  
      if(A[j] > x){  
        A[ j+1] = A[j];  
        j--; }else{break;}  
    }//endwhile  
    A[ j+1] = x;  
  }//endfor  
}
```



- 将2到n部分的n-1个元素，逐个**插入**到前面的有序段中
- 最终实现整个数组有序

插入排序 



```

MergeY4(A[], n)//p=q=1,r=n
{
  for(i = 2; i ≤ n; i++) {
    x = A[i];
    j = i-1;
    while (1 ≤ j) {
      if(A[j] > x){
        A[ j+1] = A[j];
        j--; }else{break;}
    }//endwhile
    A[ j+1] = x;
  }//endfor
}

```

1.6 插入排序

- 基本思想
- 算法:

算法 1.5 INSERTIONSORT

输入: n 个元素的数组 $A[1 \cdots n]$ 。

输出: 按非降序排列的数组 $A[1 \cdots n]$ 。

```
1. for  $i \leftarrow 2$  to  $n$ 
2.    $x \leftarrow A[i]$ 
3.    $j \leftarrow i - 1$ 
4.   while  $(j > 0)$  and  $(A[j] > x)$ 
5.      $A[j + 1] \leftarrow A[j]$ 
6.      $j \leftarrow j - 1$ 
7.   end while
8.    $A[j + 1] \leftarrow x$ 
9. end for
```

1.6 插入排序

- 比较次数: $(n-1)$
到 $n(n-1)/2$
- 赋值次数=比较次数+ $(n-1)$

```
InsertSort(A[], n)
{
    for(i = 2; i ≤ n; i++) {
        x = A[i];
        j = i-1;
        while (1 ≤ j) {
            if(A[j] > x){
                A[ j+1] = A[j];
                j--; }else{break;}
        }//endwhile
        A[ j+1] = x;
    }//endfor
}
```

1.5 选择排序

- 基本思想
- 算法: SelectionSort

算法 1.4 SELECTIONSORT

输入: n 个元素的数组 $A[1 \cdots n]$ 。

输出: 按非降序排列的数组 $A[1 \cdots n]$ 。

```
1. for  $i \leftarrow 1$  to  $n - 1$ 
2.    $k \leftarrow i$ 
3.   for  $j \leftarrow i + 1$  to  $n$     {查找第  $i$  小的元素}
4.     if  $A[j] < A[k]$  then  $k \leftarrow j$ 
5.   end for
6.   if  $k \neq i$  then 交换  $A[i]$  与  $A[k]$ 
7. end for
```

- 比较次数: $n(n-1)/2$,
- 赋值次数界于0与 $3(n-1)$ 之间

算法 1.4 SELECTIONSORT

输入: n 个元素的数组 $A[1 \cdots n]$ 。

输出: 按非降序排列的数组 $A[1 \cdots n]$ 。

```
1. for  $i \leftarrow 1$  to  $n - 1$ 
2.    $k \leftarrow i$ 
3.   for  $j \leftarrow i + 1$  to  $n$       {查找第  $i$  小的元素}
4.     if  $A[j] < A[k]$  then  $k \leftarrow j$ 
5.   end for
6.   if  $k \neq i$  then 交换  $A[i]$  与  $A[k]$ 
7. end for
```

1.6 插入排序

算法 1.5 INSERTIONSORT

输入： n 个元素的数组 $A[1 \cdots n]$ 。

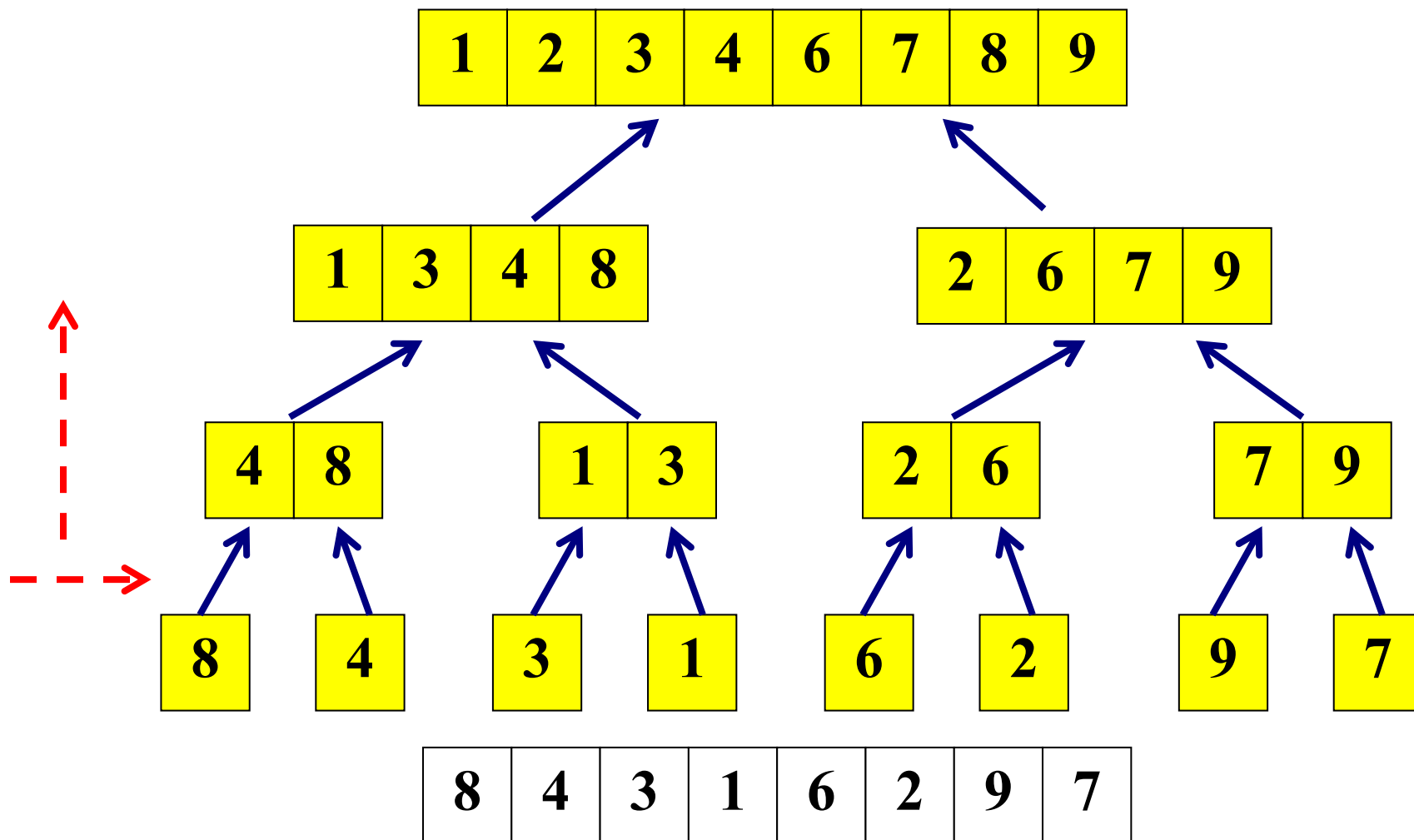
输出：按非降序排列的数组 $A[1 \cdots n]$ 。

```
InsertionSortBack(A[], n)
{
    for ( $i = n-1$ ;  $i > 0$  ;  $i--$ )
    {
         $x = A[i]$ ;
         $j = i+1$ ;
        while (( $j \leq n$ ) 且 ( $x > A[j]$ )) {
             $A[j-1] = A[j]$ ;
             $j++$ ;
        }
         $A[j-1] = x$ ;
    } //endfor
}
```

```
1. for  $i \leftarrow 2$  to  $n$ 
2.    $x \leftarrow A[i]$ 
3.    $j \leftarrow i-1$ 
4.   while ( $j > 0$ ) and ( $A[j] > x$ )
5.      $A[j+1] \leftarrow A[j]$ 
6.      $j \leftarrow j-1$ 
7.   end while
8.    $A[j+1] \leftarrow x$ 
9. end for
```

1. 7 自底向上合并排序

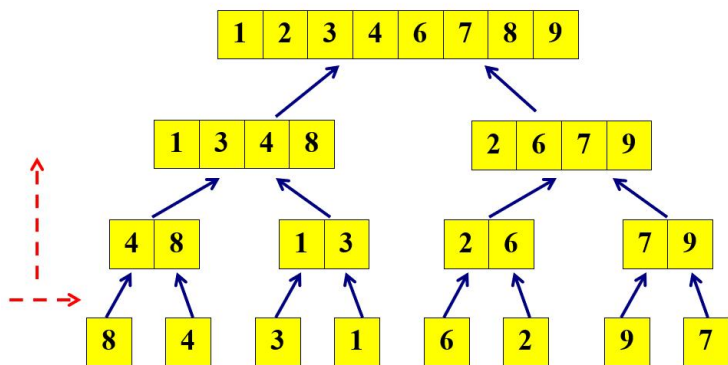
- 算法基本思想 (特殊情况 $n=2^k$)



1. 7 自底向上合并排序

- 归并排序

- 算法步骤:



```
BottomUpSort(A[], n)//假设 $n=2^k$ 
{
    生成辅助数组 B[1,...,n];
    t=1;
    while (t < n)
    {
        s = t; t = 2*t; i = 0;
        while( i+t < n){
            Merge(A, i+1, i+s, i+t);
            i = i + t; }//endwhile
        } //endwhile
    }
```

1. 7 自底向上合并排序

- 归并排序
 - 算法步骤:

```
BottomUpSort(A[], n)
{  生成辅助数组 B[1,...,n];
  t=1;
  while (t < n)
  {   s = t; t = 2*t; i = 0;
      while( i+t < n){
          Merge(A, i+1, i+s, i+t);
          i = i + t; }//endwhile
      if( i+s < n)
          Merge(A, i+1, i+s, n);
      } //endwhile
  }
```

1. 7 自底向上合并排序

以 n 等于2的 k 次幂为例(第8行不会执行)

➤ 空间开销: n

➤ 计算开销:

——赋值次数: $2nk$, 即 $2n \log n$ 次

——比较次数:

1. 7 自底向上合并排序

比较次数:

- 第1次循环, $n/2$ 次比较
- 第2次循环, $n/2$ 个2元素的有序序列被合并($n/4$ 组合并)
-
- 第 j 次循环, $n/2^j$ 组合并, 有序序列大小为 2^{j-1}
其中每组比较次数为 2^{j-1} 到 $(2^{j-1} + 2^{j-1} - 1) = (2^j - 1)$

1.7 自底向上合并排序

所以

数在 $(n/2^j)2^{j-1}$ 到 $(n/2^j)(2^j - 1)$ 之间。这样，如果令 $k = \log n$ ，那么元素的比较次数最少为

$$\sum_{j=1}^k \left(\frac{n}{2^j}\right) 2^{j-1} = \sum_{j=1}^k \frac{n}{2} = \frac{kn}{2} = \frac{n \log n}{2}$$

最多为

$$\begin{aligned} \sum_{j=1}^k \frac{n}{2^j} (2^j - 1) &= \sum_{j=1}^k \left(n - \frac{n}{2^j} \right) \\ &= kn - n \sum_{j=1}^k \frac{1}{2^j} \\ &= kn - n \left(1 - \frac{1}{2^k} \right) \quad (\text{等式 2.11}) \\ &= kn - n \left(1 - \frac{1}{n} \right) \\ &= n \log n - n + 1 \end{aligned}$$

bottomup 时间消耗

比较次数最少为:

$$\sum_{j=1}^k \left(\frac{n}{2^j}\right) 2^{j-1} = \sum_{j=1}^k \frac{n}{2} = \frac{kn}{2} = \frac{n * \log n}{2}$$

比较次数最多为:

$$\begin{aligned} \sum_{j=1}^k \frac{n}{2^j} (2^j - 1) &= \sum_{j=1}^k \left(n - \frac{n}{2^j}\right) \\ &= kn - n \sum_{j=1}^k \frac{1}{2^j} \\ &= kn - n \left(1 - \frac{1}{2^k}\right) \\ &= kn - n \left(1 - \frac{1}{n}\right) \\ &= n \log n - n + 1 \end{aligned}$$

1. 8 时间复杂性— 阶的增长

- 比较的次数
 - **BOTTOMUPSORT**: 最大 $n \log n - n + 1$
 - **SELECTIONSORT**: 恒定为 $n(n-1)/2$
- 两个实例比较（假设每次比较耗时 10^{-6} 秒）：
如果 $n=128$
 - $n \log n - n + 1 = 128 * 7 - 128 + 1 = 769$, 需要 0.0008 秒
 - $n(n-1)/2 = 128 * 127 / 2 = 8128$, 需要 0.008 秒

如果 $n=2^{20}$ （大约一百万量级）

BOTTOMUPSORT 需要 20 秒

SELECTIONSORT 需要 6.4 天

1. 8 时间复杂性— 阶的增长

- 下面假定每次执行用时为1纳秒 (10^{-9} 秒)

1. 8 时间复杂性— 阶的增长

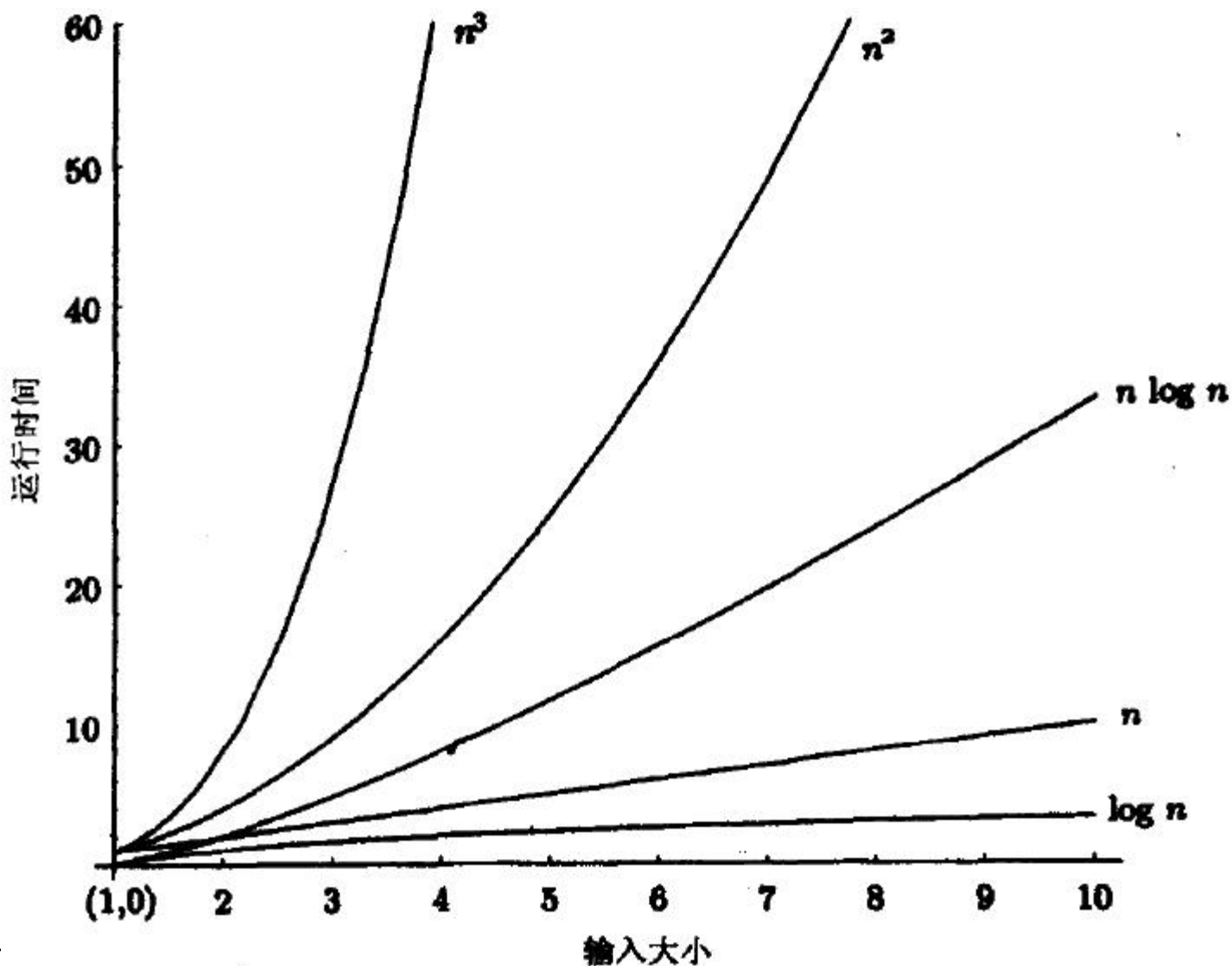


表 1.1 不同大小输入的运行时间

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
8	3 nsec	0.01 μ	0.02 μ	0.06 μ	0.51 μ	0.26 μ
16	4 nsec	0.02 μ	0.06 μ	0.26 μ	4.10 μ	65.5 μ
32	5 nsec	0.03 μ	0.16 μ	1.02 μ	32.7 μ	4.29 sec
64	6 nsec	0.06 μ	0.38 μ	4.10 μ	262 μ	5.85 cent
128	0.01 μ	0.13 μ	0.90 μ	16.38 μ	0.01 sec	10^{20} cent
256	0.01 μ	0.26 μ	2.05 μ	65.54 μ	0.02 sec	10^{58} cent
512	0.01 μ	0.51 μ	4.61 μ	262.14 μ	0.13 sec	10^{135} cent
2048	0.01 μ	2.05 μ	22.53 μ	0.01 sec	1.07 sec	10^{598} cent
4096	0.01 μ	4.10 μ	49.15 μ	0.02 sec	8.40 sec	10^{1214} cent
8192	0.01 μ	8.19 μ	106.50 μ	0.07 sec	1.15 min	10^{2447} cent
16 384	0.01 μ	16.38 μ	229.38 μ	0.27 sec	1.22 hrs	10^{4913} cent
32 768	0.02 μ	32.77 μ	491.52 μ	1.07 sec	9.77 hrs	10^{9845} cent
65 536	0.02 μ	65.54 μ	1048.6 μ	0.07 min	3.3 days	$10^{19 709}$ cent
131 072	0.02 μ	131.07 μ	2228.2 μ	0.29 min	26 days	$10^{39 438}$ cent
262 144	0.02 μ	262.14 μ	4718.6 μ	1.15 min	7 mnths	$10^{78 894}$ cent
524 288	0.02 μ	524.29 μ	9961.5 μ	4.58 min	4.6 years	$10^{157 808}$ cent
1 048 576	0.02 μ	1048.60 μ	20 972 μ	18.3 min	37 years	$10^{315 634}$ cent

注: nsec 为纳秒, μ 为毫秒, sec 为秒, min 为分钟, hrs 为小时, days 为天, mnths 为月, years 为年, cent 为世纪。

1. 8 时间复杂性— 阶的增长

元运算： 对任何计算步骤，其代价总是一个时间常量为上界，而不管输入数据或执行的算法，总称这样的计算步骤为元运算，
(基本操作)

- 算术运算：加、减、乘、除
- 比较和逻辑运算
- 赋值运算，包括变量赋值、指针赋值

1. 8 时间复杂性— O 符号

O (大 O) :

- 算法运行时间的上界 (上限)
 - 定义1.2: $f(n), g(n)$ 从自然数集合到非负实数集合的两个函数, 如果存在一个自然数 n_0 和 $c>0$,使得
$$\forall n \geq n_0, f(n) \leq c * g(n)$$
则称 $f(n)$ 为 $O(g(n))$ 。

1. 8 时间复杂性—O符号

如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在, 那么

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq \infty \Rightarrow f(n) = O(g(n))$$

通俗的说, f 没有 g 的某个常数倍增长的快。

$$f(n) = 5n^3 + 7n^2 - 2n + 13, \quad f(n) = 5n^3 + O(n^2)$$

1. 8 时间复杂性— Ω 符号

Ω :

- 算法运行时间的下界 (下限)
 - 定义1.3: $f(n), g(n)$ 从自然数集合到非负实数集合的两个函数, 如果存在一个自然数 n_0 和 $c>0$,使得
$$\forall n \geq n_0, f(n) \geq c * g(n)$$
则称 $f(n)$ 为 $\Omega(g(n))$ 。

1. 8 时间复杂性— Ω 符号

如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在, 那么

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0 \Rightarrow f(n) = \Omega(g(n))$$

通俗的说, f 的增长至少和 g 的某个常数倍一样。

➤ $f(n) = \Omega(g(n))$, 当且仅当 $g(n) = O(f(n))$

1. 8 时间复杂性— Θ 符号

Θ :

- 同时给出上界和下界
- 定义1.4: $f(n), g(n)$ 从自然数集合到非负实数集合的两个函数, 如果存在自然数 n_0 和 $c_1, c_2 > 0$, 使得
$$\forall n \geq n_0, c_1 * g(n) \leq f(n) \leq c_2 * g(n)$$
则称 $f(n)$ 为 $\Theta(g(n))$ 。

1.8 时间复杂性 — Θ 符号

如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在且大于0, 那么

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \Rightarrow f(n) = \Theta(g(n))$$

通俗的说, f 的增长至少和 g 的某个常数倍一样。

1. 8 时间复杂性— 判断方法

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq \infty \Rightarrow f(n) = O(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0 \Rightarrow f(n) = \Omega(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \Rightarrow f(n) = \Theta(g(n))$$

1. 8 时间复杂性— 判断方法

- 一些例子:

$$(1) \quad \sum_{i=0}^k a_i n^i$$

$$(2) \quad f(n) \text{ 与 } f(n+1): \quad n^2, (n+1)^2; \quad 2^n, 2^{n+1}; \\ n!, (n+1)!$$

$$(3) \quad \sum_{j=1}^n \log j$$

A.7.1 求和的积分近似

- 求以下和式的上下界

$$\sum_{j=m}^n f(j)$$

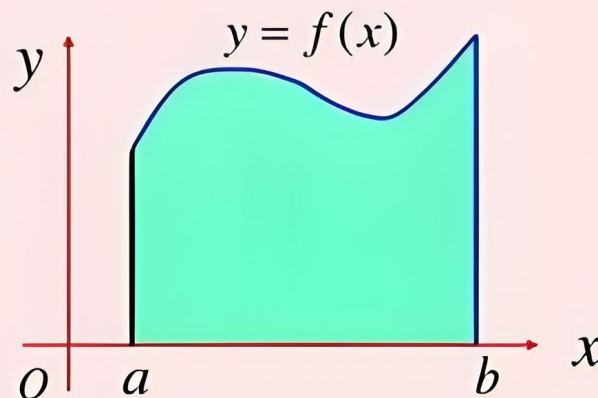
A.7.1 求和的积分近似

- 求以下和式的上下界

定积分的定义： 即 $\int_a^b f(x)dx = \lim_{n \rightarrow \infty} \sum_{i=1}^n \frac{b-a}{n} \cdot f(\xi_i)$

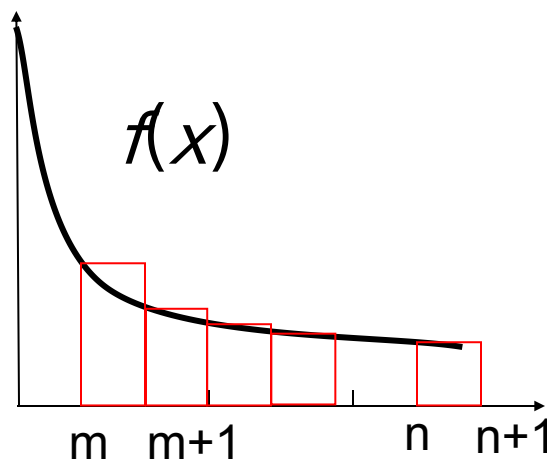
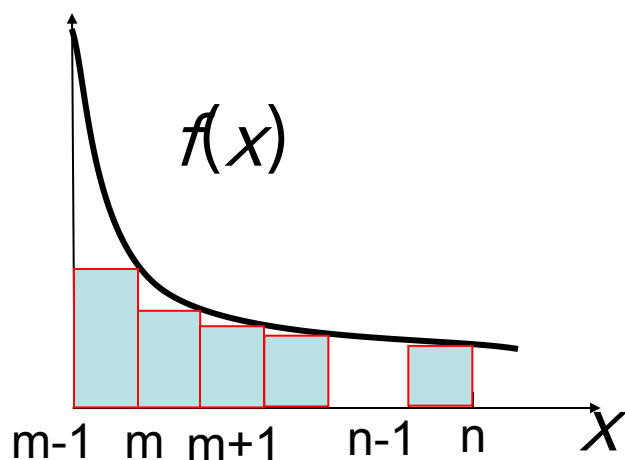
定积分的相关名称：

$\int$ —— 叫做积分号，
 $f(x)$ —— 叫做被积函数，
 $f(x)dx$ —— 叫做被积表达式，
 x —— 叫做积分变量，
 a —— 叫做积分下限，
 b —— 叫做积分上限，
 $[a, b]$ —— 叫做积分区间。



A.7.1 求和的积分近似

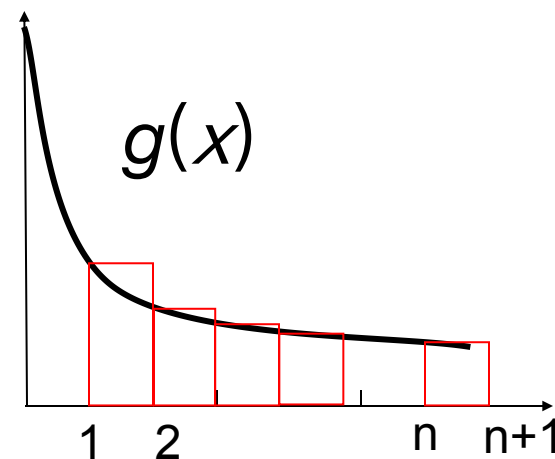
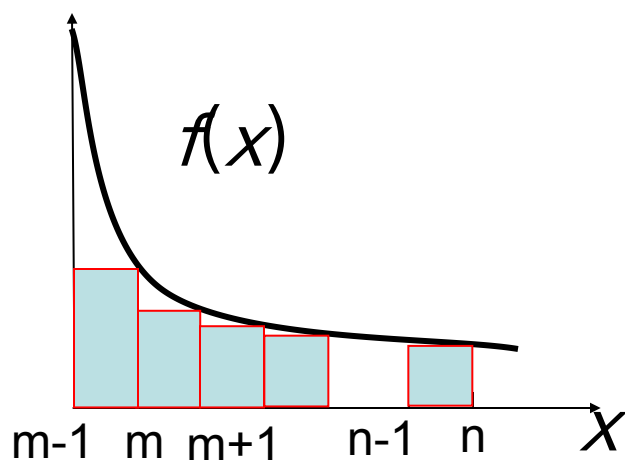
- 如果 $f(x)$ 单调递减



$$\sum_{j=m}^n f(j) \leq \int_{m-1}^n f(x) dx$$

A.7.1 求和的积分近似

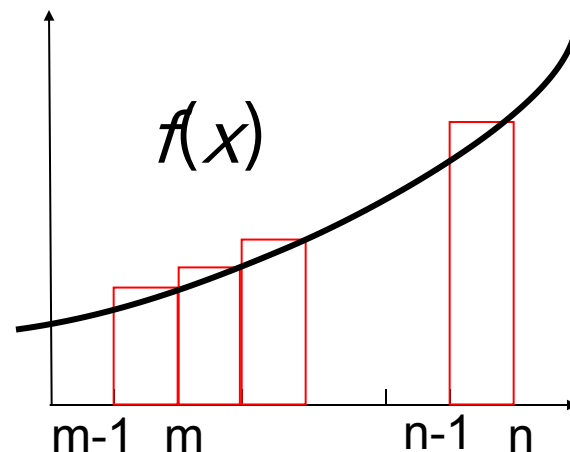
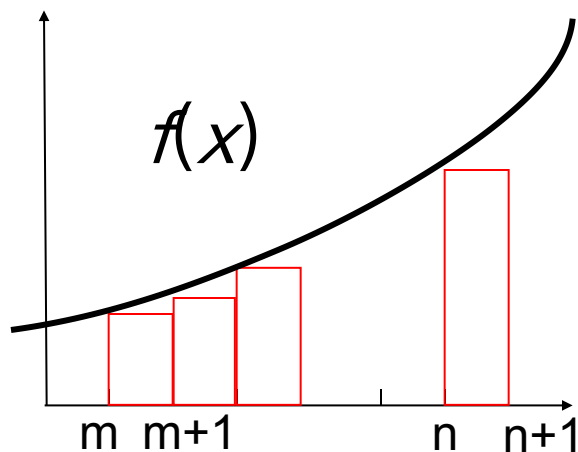
- 如果 $f(x)$ 单调递减



$$\sum_{j=m}^n f(j) \leq \int_{m-1}^n f(x) dx$$

A.7.1 求和的积分近似

- 如果 $f(x)$ 是递增的,



$$\sum_{j=m}^n f(j) \leq \int_m^{n+1} f(x) dx$$

A.7.1 求和的积分近似

- 如果 $f(x)$ 是递增的,

$$\int_{m-1}^n f(x)dx \leq \sum_{j=m}^n f(j) \leq \int_m^{n+1} f(x)dx$$

- 如果 $f(x)$ 是递减的,

$$\int_m^{n+1} f(x)dx \leq \sum_{j=m}^n f(j) \leq \int_{m-1}^n f(x)dx$$

A.7.1 求和的积分近似

- 例1, $f(n) = \sum_{j=1}^n j^k, \quad k > 1$

由于 j^k 是递增的, 我们有

$$\int_0^n x^k dx \leq \sum_{j=1}^n j^k \leq \int_1^{n+1} x^k dx$$

$$\frac{n^{k+1}}{k+1} \leq \sum_{j=1}^n j^k \leq \frac{(n+1)^{k+1} - 1}{k+1}$$

$$\Rightarrow \sum_{j=1}^n j^k = \Theta(n^{k+1})$$

A.7.1 求和的积分近似

• 例2,
$$f(n) = \sum_{j=1}^n \frac{1}{j},$$

若 $f(x)$ 递减, 则
$$\int_m^{n+1} f(x)dx \leq \sum_{j=m}^n f(j) \leq \int_{m-1}^n f(x)dx$$

但是…… $\int \frac{1}{x} dx = \ln x + C$, 而 $\ln 0$ 无意义!

$$f(n) = \sum_{j=1}^n \frac{1}{j} = 1 + \sum_{j=2}^n \frac{1}{j},$$

$$\Rightarrow f(n) = \cdot \cdot (\log n)$$

A.7.1 求和的积分近似

• 例3,
$$f(n) = \sum_{j=1}^n \log j$$

$$\int \log x \, dx = x \log x - x \log e$$

$$f(n) = \sum_{j=1}^n \log j = \sum_{j=2}^n \log j$$

$$\int_{m-1}^n f(x) dx \leq \sum_{j=m}^n f(j) \leq \int_m^{n+1} f(x) dx$$

$$\Rightarrow f(n) = \Theta(n \log n)$$

A.7.1 求和的积分近似

$$\int \log x \, dx = x \log x - x \log e + C$$

- 例4,

➤ 已知

$$f(n) = \sum_{j=1}^n \log\left(\frac{n}{j}\right)$$

➤ 证明:

$$f(n) = O(n)$$

A.8 递推关系

$$f(n) = a * f(n - 1)$$

$$f(n) = a_1 f(n - 1) + a_2 f(n - 2)$$

A.8 递推关系

- 假设, $f(n) = a_1 f(n-1) + a_2 f(n-2)$
- 特征方程 $x^2 - a_1 x - a_2 = 0$
- 假设根为 r_1 和 r_2 , 那么
$$\begin{cases} f(n) = c_1 r_1^n + c_2 r_2^n, & \text{if } r_1 \neq r_2 \\ f(n) = c_1 r^n + c_2 * n * r^n, & \text{if } r_1 = r_2 \end{cases}$$

例1

已知 $f(n) = 3f(n-1) + 4f(n-2)$ 并且 $f(0) = 1, f(1) = 4$

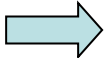
解：特征方程为 $x^2 - 3x - 4 = 0 \implies x_1 = -1, x_2 = 4$

$$\therefore f(n) = c_1(-1)^n + c_2 4^n \implies \begin{cases} f(0) = 1 = c_1 + c_2 \\ f(1) = 4 = -c_1 + 4c_2 \end{cases}$$

$$\implies c_1 = 0, c_2 = 1 \implies f(n) = 4^n$$

例2

已知 $f(n) = 2f(n-1) - f(n-2)$ 并且 $f(0) = 1, f(1) = 3$

解：特征方程为 $x^2 - 2x + 1 = 0$  $x_1 = x_2 = 1$

$$\therefore f(n) = c_1 1^n + c_2 * n * 1^n = c_1 + c_2 n$$

$$\begin{array}{l} \text{→} \left\{ \begin{array}{l} f(0) = 1 = c_1 \\ f(1) = 3 = c_1 + c_2 \end{array} \right. \end{array}$$

$$\text{→ } c_1 = 1, c_2 = 2 \quad \text{→ } f(n) = 2 * n + 1$$

例3

已知 $f(n) = f(n-1) + f(n-2)$ 并且 $f(1) = f(2) = 1$

解：特征方程为 $x^2 - x - 1 = 0 \implies x_1 = \frac{1+\sqrt{5}}{2}, x_2 = \frac{1-\sqrt{5}}{2}$

$$\therefore f(n) = c_1 \left(\frac{1+\sqrt{5}}{2} \right)^n + c_2 \left(\frac{1-\sqrt{5}}{2} \right)^n$$

$$\implies c_1 = \frac{1}{\sqrt{5}}, c_2 = -\frac{1}{\sqrt{5}}$$

$$\implies f(n) = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n$$

A1.2: 关系和等价关系

- 笛卡尔积 $A \times B = \{(a, b) | a \in A, b \in B\}$
- 关系 $R \subseteq A \times B$
- 其中, 可能 $A=B$, 也可能 $A \neq B$

A1.2: 关系和等价关系

- 关系举例

➤ 师生关系: $A = \{\text{所有老师}\}$, $B = \{\text{所有学生}\}$

$R = \{(a, b) \mid a \in A, b \in B, \text{且} b \text{听过} a \text{的课}\}$



A1.2: 关系和等价关系

- 关系举例

➤ 邻国关系: $A=B=\{\text{所有国家}\}$

$R=\{(a, b) \mid a, b \in A, \text{且从} a \text{不经过其他国家可到达} b\}$



A1.2: 关系和等价关系

- 关系举例

$F = \{\text{所有从自然数集合到非负实数集合的函数}\}$

$$R1 = \{(f, g) \mid f, g \in F, \text{ 且 } f = O(g)\}$$

$$R2 = \{(f, g) \mid f, g \in F, \text{ 且 } f = \Omega(g)\}$$

$$R3 = \{(f, g) \mid f, g \in F, \text{ 且 } f = \Theta(g)\}$$

A1.2 : 关系和等价关系

- 一种特殊情况

$$A = B, \quad R \subseteq A \times A$$

- 等价关系 (自反、对称、传递)

$$\forall a \in A, (a, a) \in R$$

$$\forall a, b \in A, \text{如果 } (a, b) \in R, \text{则 } (b, a) \in R$$

$$\forall a, b, c \in A, \text{如果 } (a, b) \in R, (b, c) \in R, \text{则 } (a, c) \in R$$

1. 8 时间复杂性 — Θ 符号

– $f(n)=\Theta(g(n))$, 当且仅当 $f(n)=O(g(n))$ 且 $f(n)=\Omega(g(n))$

– 而且 $g(n)=\Theta(f(n))$ 对称
 $f(n)=\Theta(f(n))$ 自反

– 如果 $g(n)=\Theta(h(n))$

– 那么 $f(n)=\Theta(h(n))$ 传递

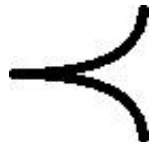
$2n^2, 3n^2-2n+5$

$\Rightarrow$ 等价关系 $\Rightarrow$ 等价类 n^2

1.8 时间复杂性— o 符号

小 o :

- 为了表示复杂性等价类之间的关系



- $1, \log \log n, \log n, n, n^2, 2^n, n!$

1. 8 时间复杂性— o 符号

– 定义1.5: $f(n), g(n)$ 从自然数集合到非负实数集合的两个函数, 如果存在自然数 n_0 和 $c > 0$, 使得

$$\forall n \geq n_0, f(n) < c * g(n)$$

则称 $f(n)$ 为 $o(g(n))$ 。

如果 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ 存在, 那么

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f(n) = o(g(n))$$

1. 8 时间复杂性—小结

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq \infty \Rightarrow f(n) = O(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \neq 0 \Rightarrow f(n) = \Omega(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \Rightarrow f(n) = \Theta(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f(n) = o(g(n))$$

算法复杂度分析的意义

- 已知待求解问题的多种算法时，**挑选**复杂度尽可能低的算法进行应用。
- 给定待求解的问题，**设计**复杂度尽可能低的算法。
- 设计出算法后，不要急于实现，而是先进行复杂度**分析**后；若该算法确实可行，才有实现的价值与必要。

几点说明

- “低阶复杂度的算法比高阶复杂度的算法效率高” 这个结论，只是在问题的规模充分大时才成立。
 - 复杂度分别为 n^3 与 $10n^2$ ：当 $n > 10$ 时， $n^3 > 10n^2$ ；而 $n < 10$ 时， $n^3 < 10n^2$ ，即复杂度为 n^3 的算法更有效。
- 在问题规模较小时，我们往往并不一味追求低复杂度的算法，而是更侧重于算法的简单性。
- 当两个算法的渐近复杂度的阶相同时，必须进一步综合考察渐近复杂度表达式中的低阶及常数因子才能判别它们的优劣。
 - 例如：复杂度为 $n \log n / 100$ 的算法显然比复杂度为 $100n \log n$ 的算法来得有效。

1.9 空间复杂性

- $T(n)$
- $S(n)$: 执行计算的内存开销
- $S(n) = O(T(n))$
- 几个例子: P20

1.10 最优算法

- 如果可以证明任何一个求解问题 Π 的算法必定是 $\Omega(f(n))$,
- 那么把在 $O(f(n))$ 时间内求解问题 Π 的任何算法都称为问题 Π 的最优算法

1.11 估计算法运行时间

- 1.11.1 计算迭代次数
- 1.11.2 计算**基本运算**的频度

如果一个元运算的频度的常数倍大于其他任何元运算的频度，则该元运算称为基本运算。

比如，Merge中的元素赋值

- 1.11.3 使用递推关系（以后会碰到）

1.12 最坏情况和平均情况

根据输入的不同状况

- 最坏情况
- 平均情况
- 最好情况

每种情况对应的复杂度计算结果，都有上界和下界

1.13 平摊分析

• P29

例 1.32 考虑下面的问题。有一个双向链表(见 3.2 节), 初始时由一个节点组成, 节点内容是整数 0。有一个 n 个正整数的数组 $A[1 \cdots n]$ 作为输入, 我们用如下方法处理它们: 如果当前的 x 是奇数, 那么将 x 添加到表中; 如果 x 是偶数, 先添加 x 到表上, 然后移去表中所有在 x 之前的奇数元素。这个问题的算法概况如下, 并在图 1.7 中说明。输入为

5	7	3	4	9	8	7	3
---	---	---	---	---	---	---	---

最坏情况: 除了最后一个, 每个数都被删除

```
for( $j=1; j \leq n; j++$ ) {
```

```
     $x=A[j]$ ;    将 $x$ 加入表中;
```

```
    if( $x$ 是偶数) { while( $\text{pred}(x)$ 为奇数) 删除 $\text{pred}(x)$ ;
```

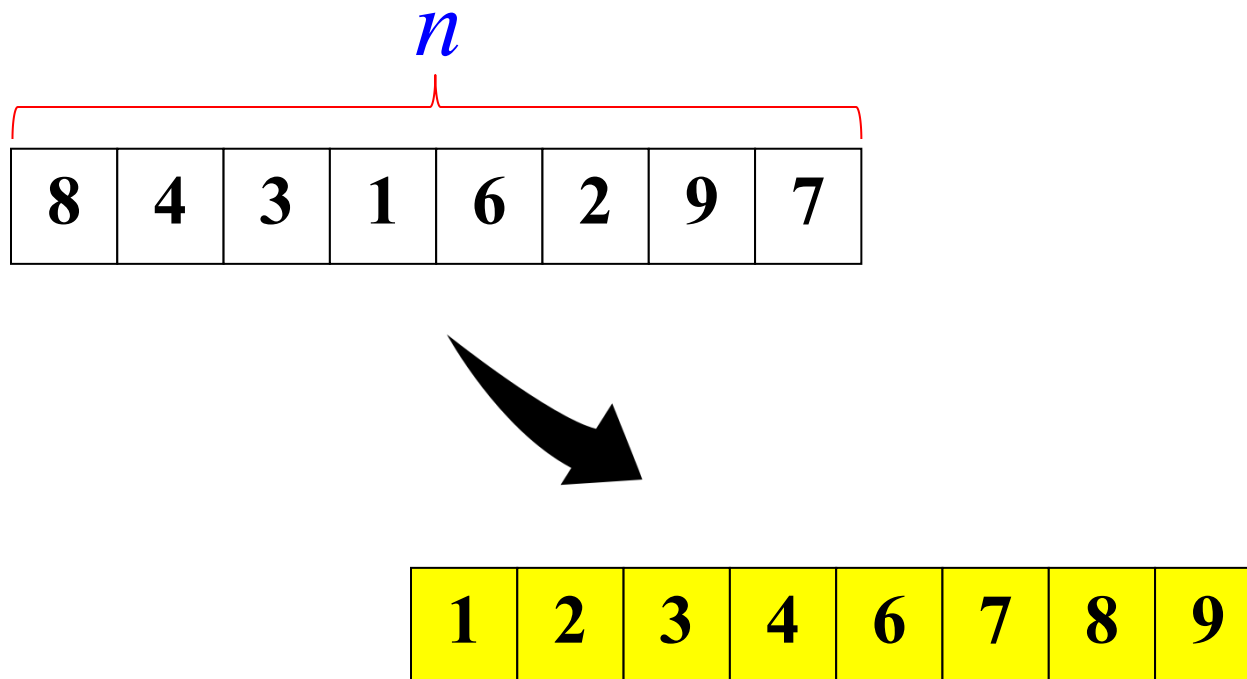
每轮while循环的次数?

```
    }
```

2026-5-18

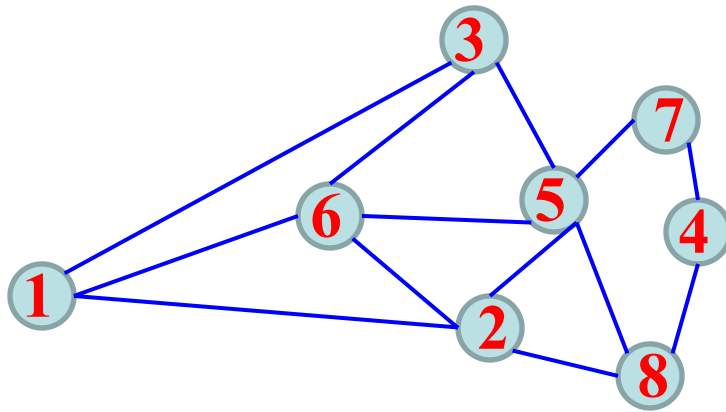
1.14 输入大小和问题实例

- **排序和搜索问题**: 用数组或表中**元素的个数**作为输入大小;



1.14 输入大小和问题实例

- **图的算法**：输入大小通常指图中的边或顶点的数目，或二者皆有；



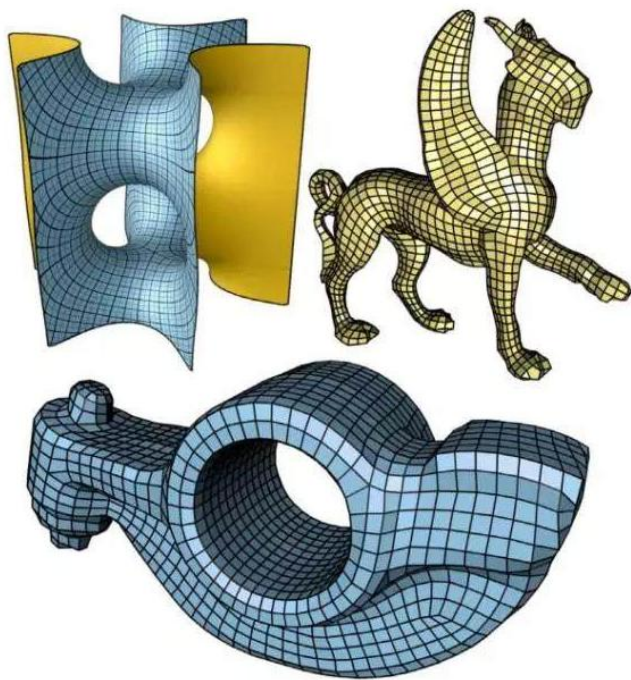
$$G=(V, E)$$

$$|V| = n$$

$$|E| = m$$

1.14 输入大小和问题实例

- **计算几何**：算法的输入大小通常用点、顶点、边、线段或多边形等的个数来表示；

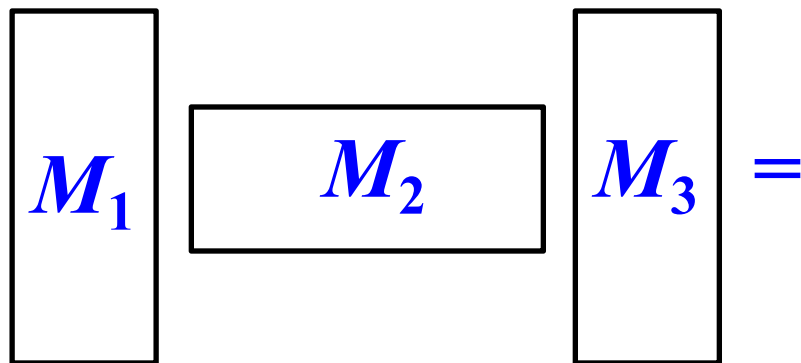


1.14 输入大小和问题实例

- **矩阵运算**: 输入大小通常是输入矩阵的维数;

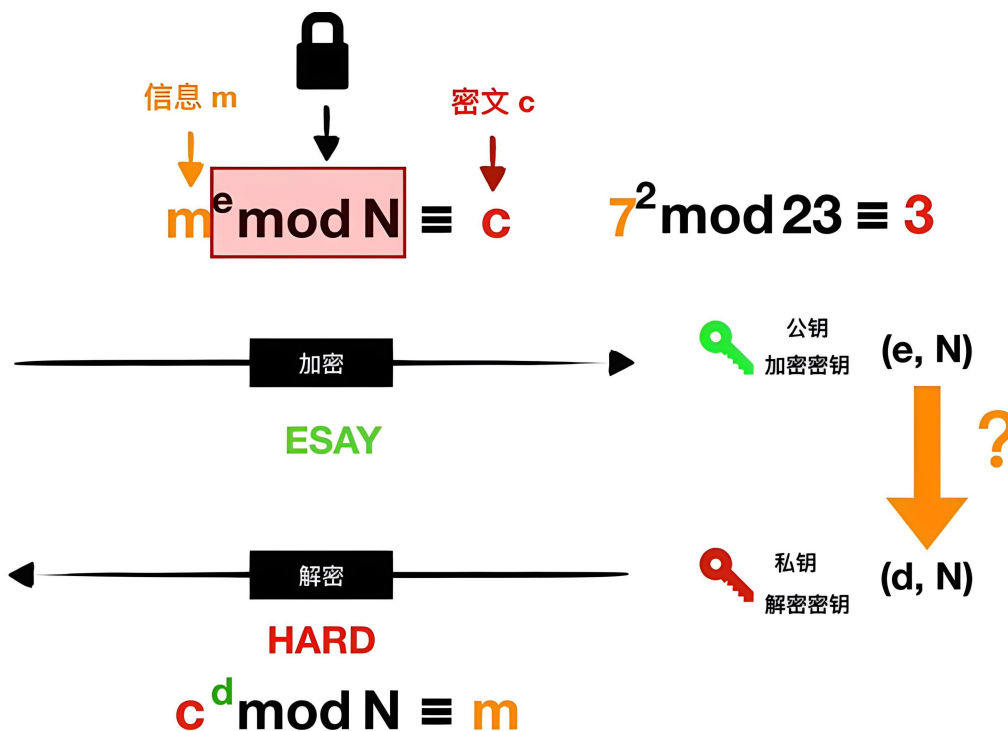
计算 $M_1 M_2 M_3$

其维度分别是: $m \times n, n \times n, n \times m$



1.14 输入大小和问题实例

- **数论算法和密码学**: 通常选择输入的比特数来表示它的长度



$$N = (b_k \dots b_2 b_1 b_0)_2$$

$$N = \sum_{i=0}^k b_i 2^i = \Theta(2^k)$$

第1章的课后作业

• 作业: 1.14, 1.15, 1.16, 1.38

1.14 用 true 或 false 填空。

$f(n)$	$g(n)$	$f = O(g)$	$f = \Omega(g)$	$f = \Theta(g)$
$2n^3 + 3n$	$100n^2 + 2n + 100$			
$50n + \log n$	$10n + \log \log n$			
$50n \log n$	$10n \log \log n$			
$\log n$	$\log^2 n$			
$n!$	5^n			

1.15 用 Θ 符号表示下列函数:

- (a) $2n + 3 \log^{100} n$ (b) $7n^3 + 1000n \log n + 3n$
(c) $3n^{1.5} + (\sqrt{n})^3 \log n$ (d) $2^n + 100^n + n!$

1.16 用 Θ 符号表示下列函数:

- (a) $18n^3 + \log n^8$ (b) $(n^3 + n)/(n + 5)$
(c) $\log^2 n + \sqrt{n} + \log \log n$ (d) $n!/2^n + n^n$

1.38 请设计一个算法, 对于给定的 x 的值, 在时间

- (a) $\Omega(n^2)$ (b) $O(n)$

求输入多项式

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

的值。

• 3月27日 (周五) 上课前, 交作业。